CYBERSECURITY HOMELAB REPORT · SOC ANALYSIS

Brute-Force Attack Detection and Investigation

A dual-SIEM lab using Wazuh, Splunk, Active Directory and endpoint telemetry to detect, correlate and investigate a credential brute-force attack, with a Sigma rule-portability test.

Author	Ishola Sunday Bidemi

Lab dates	5, 7 and 14 October 2026 (Africa/Lagos, UTC+01:00)
Version	1.0 · 14 October 2026
Environment	VirtualBox host-only lab · Ubuntu (Wazuh manager/indexer and Splunk) · Windows Server 2019 domain controller · Windows 10 domain client · Kali Linux attacker box
Technique	MITRE ATT&CK T1110 (Brute Force), credential access
Classification	Educational lab. The attack targeted a lab-created domain account on an isolated host-only network with no internet-facing exposure.

Prepared from the author's own lab screenshots, terminal output and configuration excerpts. Original screenshots are retained unchanged as source evidence; figures in this report are cropped, contrast-adjusted copies. No evidence content has been altered.

Table of Contents

1. Executive Summary

2. Introduction and Objectives

- 2.1 Background and motivation
- 2.2 Scope
- 2.3 Objectives and success criteria

3. Technical Background

- 3.1 From a logon attempt to a dashboard alert
- 3.2 Windows Event ID 4625 and why the sub-status matters
- 3.3 How the Wazuh correlation rule works
- 3.4 What Sigma is and why the conversion failure matters
- 3.5 MITRE ATT&CK technique used

4. Lab Environment and Architecture

- 4.1 Machines and addresses
- 4.2 Architecture and data flow
- 4.3 Software and versions

5. Prerequisites and Safety

- 5.1 Required resources
- 5.2 Safety and scope controls
- 5.3 Backups and recovery

6. Step-by-Step Implementation

- 6.1 Phase A: Preparing the attacker (Kali Linux)
- 6.2 Phase B: Verifying the Ubuntu SIEM host (VM1)
- 6.3 Phase C: Configuring the Windows 10 agent (VM2)
- 6.4 Phase D: Installing Sigma on Kali
- 6.5 Phase E: Running the brute-force attack
- 6.6 Phase F: Writing the Sigma detection rule
- 6.7 Phase G: Deploying the Wazuh and Splunk detections
- 6.8 Phase H: Troubleshooting and live validation

7. Configuration and Detection Logic

- 7.1 The Wazuh rules (as deployed)
- 7.2 Line-by-line explanation
- 7.3 Rule inventory and status
- 7.4 The Splunk SPL query
- 7.5 The Sigma rules

8. Testing and Results

- 8.1 Test matrix
- 8.2 What succeeded, what failed, what remains unverified

9. Evidence and Screenshot Analysis

- 9.1 Image preparation and evidence integrity
- 9.2 Wazuh dashboard evidence

10. Troubleshooting and Root-Cause Analysis

- 10.1 Issue 1: early attack attempts rejected at the SMB layer

10.2 Issue 2: Sigma correlation rule does not convert to the OpenSearch backend
10.3 Issue 3: two different IP addresses recorded for the Ubuntu SIEM host
10.4 Issue 4: duplicate/overlapping screenshots

11. Security Findings and Risk Analysis

12. Mitigation and Remediation

13. What I Learned

14. Practical Knowledge and Competencies
14.1 Recommended next exercises

15. Reproduction Guide
15.1 Part 0: base infrastructure setup
15.2 Part A: run the attack
15.3 Part B: verify in Wazuh
15.4 Part C: verify in Splunk
15.5 Troubleshooting
15.6 Cleanup and rollback

16. Limitations and Future Improvements

17. Conclusion

Appendix A: Evidence Register

Appendix B: Full Command Reference (Author's Step-by-Step Replication Log)

Appendix C: Glossary

List of Figures

Figure 1. Laboratory architecture and data flow
Figure 2. Kali terminal: wordlists, housekeeping and the CrackMapExec launch
Figure 3. Early attack output: non-existent accounts rejected at the connection level
Figure 4. Continued cycling through non-existent accounts
Figure 5. Attack run reaching the end of the non-existent-account list
Figure 6. Wazuh local_rules.xml open in nano: the built-in rules above the custom ones
Figure 7. Rules 100100 and 100101 as deployed
Figure 8. Rule 100101 scrolled fully into view, with its closing tags
Figure 9. Sigma correlation-rule conversion failing, and the single-event rule succeeding
Figure 10. Event log: the escalation pattern, page 2 of 667
Figure 11. Event log: earliest visible page, with the 4625/4624 context
Figure 12. Dashboard overview: alert-group evolution and alert-level timeline
Figure 13. Dashboard summary tiles: the headline counts
Figure 14. Top 5 alerts, rule groups and PCI DSS requirements

List of Tables

Table 1. Objectives, success criteria and outcome
Table 2. The detection pipeline and where each stage was checked
Table 3. ATT&CK mapping
Table 4. Lab components
Table 5. Software versions and tools observed
Table 6. CrackMapExec command anatomy
Table 7. Rule elements explained
Table 8. Custom and relevant built-in rules
Table 9. SPL query anatomy
Table 10. Test scenarios and verification status
Table 11. Issue 1 analysis
Table 12. Issue 2 analysis
Table 13. Findings and confidence
Table 14. Mitigations
Table 15. Skills demonstrated, with the evidence for each
Table 16. Reproduction checklist
Table 18. Evidence register: every figure and its source (originals kept unchanged)

Abbreviations and Acronyms

AD / ADDS	Active Directory / Active Directory Domain Services
ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge, a catalogue of attacker behaviour
DC	Domain Controller
MFA	Multi-Factor Authentication
NTLM	NT LAN Manager, a Windows authentication protocol
PCI DSS	Payment Card Industry Data Security Standard (referenced here only as a Wazuh rule-group tag)
SIEM	Security Information and Event Management
SMB	Server Message Block, the Windows file/authentication-sharing protocol used for the attack
SOC	Security Operations Centre
SPL	Search Processing Language, Splunk's query language
Sysmon	Microsoft System Monitor, an optional endpoint telemetry driver (installed but not the detection source used)
WAT	West Africa Time, UTC+01:00 (the lab's local time zone)

How to Use This Report

This report can be read in two ways. Choose the path that matches what you want.

If you want to...	Read
Rebuild the lab and get the same detections	Chapter 5 (prerequisites and safety) → Chapter 15 (Reproduction Guide, with a checklist) → Chapter 7 (the rules and SPL). Use Chapter 10 only if something fails.
Understand how the attack was run and caught	Chapter 3 (pipeline) → Chapter 6 (steps in order, each with a Goal and a Checkpoint) → Chapter 8 (results).
Judge the result and the evidence	Chapter 1 (summary) → Chapter 8 (test results) → Chapter 9 (evidence) → Chapter 16 (limitations).
Reuse the Sigma rules or SPL query	Chapter 7, then Chapter 10 (the Sigma correlation-conversion failure).

Conventions

- **Figure N** and **Table N** numbers are the same everywhere; the contents pages list the page of each.
- Commands appear in dark boxes. Run them exactly as shown, one at a time.
- Machine roles: **VM1** = Ubuntu, Wazuh manager/indexer and Splunk (192.168.56.101/.102); **VM2** = Windows 10 domain client, DESKTOP-T6T1N2V (192.168.56.116); **VM3** = Windows Server 2019 domain controller, HopeLabs.Local (192.168.56.115); **VM4** = Kali Linux attacker box (192.168.56.103).
- **Checkpoint** = a result you must see before continuing.

CHAPTER 1

Executive Summary

This laboratory simulated, detected and investigated a credential brute-force attack against an Active Directory domain account, using two independent SIEM platforms side by side. CrackMapExec ran a dictionary attack over SMB against a domain user (`HR-STAFF1`) from a Kali Linux VM. Windows Security auditing on the target generated Event ID 4625 (failed logon) records, which were analysed two ways: by a custom two-stage Wazuh correlation rule on the Ubuntu manager, and by an independently built Splunk SPL query and dashboard on the same Ubuntu host. A portable Sigma rule set was also authored and partially converted to the Wazuh/OpenSearch backend, which surfaced a real limitation of that toolchain.

The security problem. Password-guessing against a domain account is one of the oldest and still most common ways into a network. A single failed logon is not alarming, but five failures against the same account inside two minutes is a different signal, and a SOC needs a rule that escalates one into the other, in more than one tool, so the detection survives a platform change.

What happened. CrackMapExec was pointed at the Windows 10 client with 1,000+ usernames and 1,000+ passwords from SecLists. Early attempts against local/administrative-sounding accounts (`msadmin` , `vagrant` , `kali` , `ubuntu` , `creativity` , `family` , `admin001`) returned `STATUS_NO_LOGON_SERVERS` , because those accounts do not exist in the domain. Once the tool reached the real domain account `HR-STAFF1` , responses changed to `STATUS_LOGON_FAILURE` (a real account, wrong password) and then, on the password `Hope199872` , a success (`[+] HopeLabs.Local\HR-STAFF1:Hope199872`). Over the 24-hour window surrounding the test, the Wazuh dashboard for the target agent recorded 22,069 total alerts, 21,898 of them level 12 or above, with 118 authentication-failure alerts and 2 authentication-success alerts.

22,069	118	2	1
total Wazuh alerts for the agent over the 24-hour window	authentication-failure alerts (rule 100100 / 100101)	authentication-success alerts recorded in the same window	domain account compromised in testing: HR-STAFF1 / Hope199872

Verified outcomes. Both custom Wazuh rules fired as designed: rule 100100 (level 10) on each failed logon, and rule 100101 (level 12, frequency 5, timeframe 120 seconds) once five failures against the same account landed inside a two-minute window. A built-in rule (92652, "Successful Remote Logon Detected") then recorded the successful NTLM logon for `HR-STAFF1` immediately after. The Splunk SPL query independently grouped the same `EventCode=4625` events by source IP and flagged the same threshold. The Sigma single-event rule converted cleanly to the Wazuh/OpenSearch backend; the Sigma *correlation* rule (the 5-in-120-seconds escalation) did not convert, because the installed Lucene backend does not support Sigma correlation rules. This is reported as a tool limitation, not a rule-authoring mistake, since the single-event rule converted without issue.

What is not independently verified in this report. The 22,069/118/2 counts and the "667 pages / ~10,000 events" figure are read from the dashboard screenshots and are reported as shown; the underlying raw alert file (`alerts.json`) was not queried with `grep` / `uniq` the way the phishing-detection lab's rules were, so there is no independent file-level count to cross-check the dashboard tiles against. Two screenshots referenced in the author's working draft of this report (a Splunk dashboard pie chart and a raw Splunk event-detail page) were not among the images supplied for this rebuild and are described from that draft text only; they are not reproduced as figures here (Chapter 16).

CHAPTER 2

Introduction and Objectives

2.1 Background and motivation

Brute-force and password-spraying attacks remain one of the most common ways attackers obtain an initial foothold, because they need no vulnerability, only a weak or reused password and an exposed authentication service. Windows Security auditing records every logon attempt as Event ID 4624 (success) or 4625 (failure), with a sub-status code that explains why a failure occurred. A SOC's job is to turn a stream of individually unremarkable 4625 events into a single, actionable alert once a pattern — many failures, one account, a short window — appears, and to do that in whichever SIEM the organisation runs.

2.2 Scope

In scope: a four-VM VirtualBox lab on a host-only network, a domain account created specifically for the test, a dictionary attack run with CrackMapExec over SMB, detection in Wazuh (custom rules) and Splunk (an SPL query and dashboard), and a Sigma rule set written to make the detection logic portable between them. Out of scope: automated response (account lockout or blocking), real multi-factor authentication, and any attack technique other than password guessing.

2.3 Objectives and success criteria

Table 1. Objectives, success criteria and outcome

#	Objective	Success criterion	Result
O1	Generate a realistic brute-force attack against a domain account	CrackMapExec returns authentication results (success or failure) rather than connection errors	**Met**: the attack reached `STATUS_LOGON_FAILURE` then success (Figures 11 and 13)
O2	Detect each failed logon	Wazuh rule 100100 fires on Event ID 4625	**Met** (118 alerts recorded for the window, Figure 13)
O3	Correlate repeated failures into a brute-force alert	Wazuh rule 100101 fires once 5 failures occur against the same account within 120 seconds	**Met**, and followed by a successful-logon alert (rule 92652) for the same account
O4	Reproduce the detection logic in Splunk, independent of Wazuh	An SPL query over `EventCode=4625` groups by source IP and flags 5+ failures	**Met** (query written and run; see Chapter 7)
O5	Make the detection logic portable with Sigma	Both the single-event and the correlation Sigma rule convert to the Wazuh/OpenSearch backend	**Partly met**: the single-event rule converted; the correlation rule did not (Chapter 10)
O6	Confirm the account was actually compromised, not just flagged	A successful logon for the attacked account appears after the failures	**Met**: `[+] HopeLabs.Local\HR-STAFF1:Hope199872` (Figure 11) and rule 92652

CHAPTER 3

Technical Background

3.1 From a logon attempt to a dashboard alert

As in any SIEM lab, "nothing in the dashboard" can mean a failure at any of several separate stages. This lab's pipeline has five stages, each independently checkable.

Table 2. The detection pipeline and where each stage was checked

Stage	Plain meaning	In this lab	How it was checked
1. Event generation	Windows records that a logon was attempted	The domain controller/target writes Event ID 4625 (failure) or 4624/92652-mapped success to the Security log	The dashboard shows both Windows Failed Logon Detected and Successful Remote Logon Detected entries (Figure 10)
2–3. Collection and forwarding	An agent reads the log and sends it to the SIEM	The Wazuh agent on the target forwards Security events to the manager on VM1 (192.168.56.102); the Splunk Universal Forwarder does the same to the Splunk indexer	Alerts carry `agent.name` `DESKTOP-T6T1N2V`; Splunk events show the matching workstation
4. Rule evaluation	Rules compare fields against conditions	Wazuh rules 100100/100101 in `local_rules.xml`; the Splunk SPL query; Sigma rules translated to each backend	`wazuh-analysisd -t`; SPL run in Search & Reporting
5. Alert generation and display	A matching rule writes an alert, shown in the dashboard	Wazuh Threat Hunting dashboard; Splunk SOC Brute Force Detection dashboard	Figures 12, 13 and 14

3.2 Windows Event ID 4625 and why the sub-status matters

Event ID 4625 is written whenever a Windows logon fails. Its `Status` and `Sub Status` fields explain why: a bad username produces a different code from a bad password for an account that exists. This matters for interpreting the attack's early phase here: many of CrackMapExec's first attempts returned the SMB-level error `STATUS_NO_LOGON_SERVERS` rather than a Windows 4625 event at all, because those usernames (`msadmin`, `vagrant`, `kali`, `ubuntu`, and others) do not exist as domain accounts; a non-existent account does not generate the same audit trail as a real one receiving a wrong password.

3.3 How the Wazuh correlation rule works

A Wazuh rule has an ID, a severity level (0–15) and conditions. `<if_sid>` makes a rule a child of another rule in the Sysmon/Windows event rule chain. `frequency` and `timeframe` turn a rule into a correlation rule that fires after a given number of matches in a given number of seconds, and `<if_matched_sid>` names the single rule ID whose matches are counted (Chapter 7 gives the full syntax).

3.4 What Sigma is and why the conversion failure matters

Sigma is a vendor-neutral format for writing a detection once and converting it to the query language of several SIEM backends (Splunk, Wazuh/OpenSearch, and others), via the `sigma-cli` tool and backend plugins. A Sigma rule can be a plain single-event rule, or a `correlation` rule that references one or more single-event rules and adds a grouping field, a time window and a threshold — exactly the shape of this lab's brute-force detection. Not every backend plugin supports every Sigma feature; this lab's Lucene/OpenSearch backend plugin, as installed, does not support Sigma correlation rules, and the CLI says so explicitly rather than producing a wrong or silent result. This is documented here as a concrete, evidenced example of a portability limitation, not as something this lab fixed.

3.5 MITRE ATT&CK technique used

Table 3. ATT&CK mapping

Technique	Name	Where it applies	Why it matters
T1110	Brute Force	Rules 100101, both Sigma rules, the Splunk query	The core technique catalogued for password-guessing attacks against a single account

CHAPTER 4

Lab Environment and Architecture

4.1 Machines and addresses

Table 4. Lab components

VM	Role	OS	IP address
VM1	Wazuh manager/indexer and Splunk indexer	Ubuntu	192.168.56.102 (also referenced as .101 for the Wazuh dashboard URL)
VM2	Target system hosting the AD user HR-STAFF1 (DESKTOP-T6T1N2V)	Windows 10	192.168.56.116
VM3	Domain controller hosting Active Directory (HopeLabs.Local)	Windows Server 2019	192.168.56.115
VM4	Attacker machine: Sigma CLI, CrackMapExec, wordlists (host "Creativity")	Kali Linux	192.168.56.103

Evidence gap. The draft report gives VM1's IP as both .101 and .102 in different places (Wazuh dashboard reached at .101, Splunk and the agent enrollment target at .102). Both addresses point at the same Ubuntu host in this lab; the discrepancy is most likely two interfaces or a typo in the original notes, and it is carried through here as recorded rather than silently resolved.

Host machine: Dell Latitude 7290. Network adapters on all VMs: Adapter 1 = NAT, Adapter 2 = Host-only (VirtualBox's default host-only network, 192.168.56.0/24).

4.2 Architecture and data flow

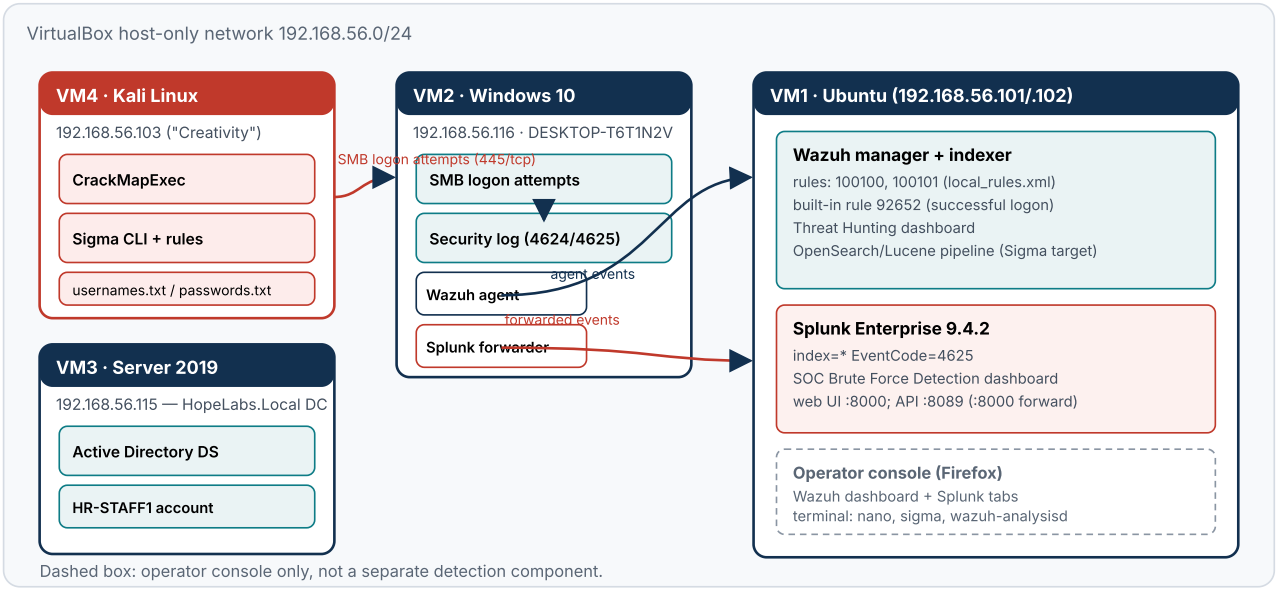


Figure 1. Laboratory architecture and data flow

Step: Chapter 4 **Source:** drawn for this report from the evidence listed in Chapter 4

Four VMs on one VirtualBox host-only network: Kali (VM4) attacks the Windows 10 client (VM2), which authenticates against the Server 2019 domain controller (VM3); the client's Security log is forwarded by two independent paths — a Wazuh agent and a Splunk Universal Forwarder — to the Ubuntu host (VM1), which runs both SIEMs.

4.3 Software and versions

Table 5. Software versions and tools observed

Software	Version / detail	Where seen
Oracle VirtualBox	Host application for all four VMs	Lab setup
Splunk Enterprise	9.4.2 (shown in the Splunk Search & Reporting app)	Figure of raw-event search, per the author's working draft
Wazuh	Indexer, manager and dashboard on VM1; agents on VM2 and VM3	Dashboard screenshots (Figures 13–14)
CrackMapExec	Installed via `apt install crackmapexec`; Python 3 dist-packages (impacket, etc.) emit several raw-string `SyntaxWarning` messages on startup — cosmetic, not failures	Figures 2, 3 and 4
Sigma CLI	`sigma-cli` installed via `pipx`; backend plugins for Splunk, Wazuh, OpenSearch and Sysmon installed	Figure 9
Wordlists	SecLists `usernames.txt` and `passwords.txt`, 1,000+ entries each	Figure 2

CHAPTER 5

Prerequisites and Safety

5.1 Required resources

- Host computer able to run four virtual machines concurrently (suggested specs: Ubuntu 4 vCPU/8 GB RAM/60 GB disk for Wazuh + Splunk; Windows 10 and Server 2019 each 2 vCPU/4 GB RAM; Kali 2 vCPU/4 GB RAM/40 GB disk).
- A Windows Server 2019 VM promoted to a new Active Directory forest (`HopeLabs.Local`) with at least one standard domain user account created specifically for the test (`HR-STAFF1`), with a password drawn from or close to a common wordlist so the attack can succeed inside a reasonable lab window.
- A Windows 10 VM joined to that domain, with the Wazuh agent and (optionally) the Splunk Universal Forwarder installed and enrolled.
- An Ubuntu VM with the Wazuh manager, indexer and dashboard, and Splunk Enterprise, both installed and reachable from a browser.
- A Kali Linux VM with CrackMapExec and the SecLists wordlists, and, for the Sigma portion, `sigma-cli` with the Splunk, Wazuh, OpenSearch and Sysmon backend plugins.

5.2 Safety and scope controls

- **Isolated network only.** All four VMs sit on a VirtualBox host-only network (192.168.56.0/24) with no route to the internet or to any production system. Never point CrackMapExec, or any brute-force tool, at an account or host you do not own or have explicit authorization to test.
- **A disposable test account.** `HR-STAFF1` was created solely for this exercise, with a password chosen to be guessable within the lab's time budget. Do not reuse that password or account naming pattern on any real system.
- **Expect account lockout policies to interact with this test.** This lab's domain, as configured, did not appear to lock the account out during the run (the attack continued past multiple failures to a success); a domain with a standard lockout policy would lock the account after a handful of failures, which is itself a legitimate and complementary control discussed in Chapter 12.
- **No automated response was configured or triggered.** This lab only detects; it does not block or disable the account automatically (see the recommendation in Chapter 12).

5.3 Backups and recovery

Wazuh's `local_rules.xml` is a single file; as with any hand-edited configuration, keep a copy before editing (`sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak`) and validate every change with `sudo /var/ossec/bin/wazuh-analysisd -t` before restarting the manager. The working draft of this report does not show a backup step for this particular file, so this is stated as standard practice rather than as something captured in a screenshot.

CHAPTER 6

Step-by-Step Implementation

> **Note. How to use this chapter.** It records the lab in the order the author's own methodology notes give it, grouped into lettered phases (A–H), with a Goal and a Checkpoint added to each phase so a reader knows what result to expect before moving on. If you only want a clean rebuild, follow the checklist in Chapter 15 instead.

6.1 Phase A: Preparing the attacker (Kali Linux)

Goal: have working wordlists and a confirmed network path to the target before starting the real attack.
Checkpoint: `crackmapexec --version` returns a version string and a ping to the target succeeds.

Using SecLists, built `usernames.txt` (1,000+ entries) and `passwords.txt` (1,000+ entries) for brute-forcing. Installed CrackMapExec (`sudo apt install crackmapexec -y`) and confirmed the version. Confirmed connectivity and protocol access to the Windows 10 VM before the attack run.

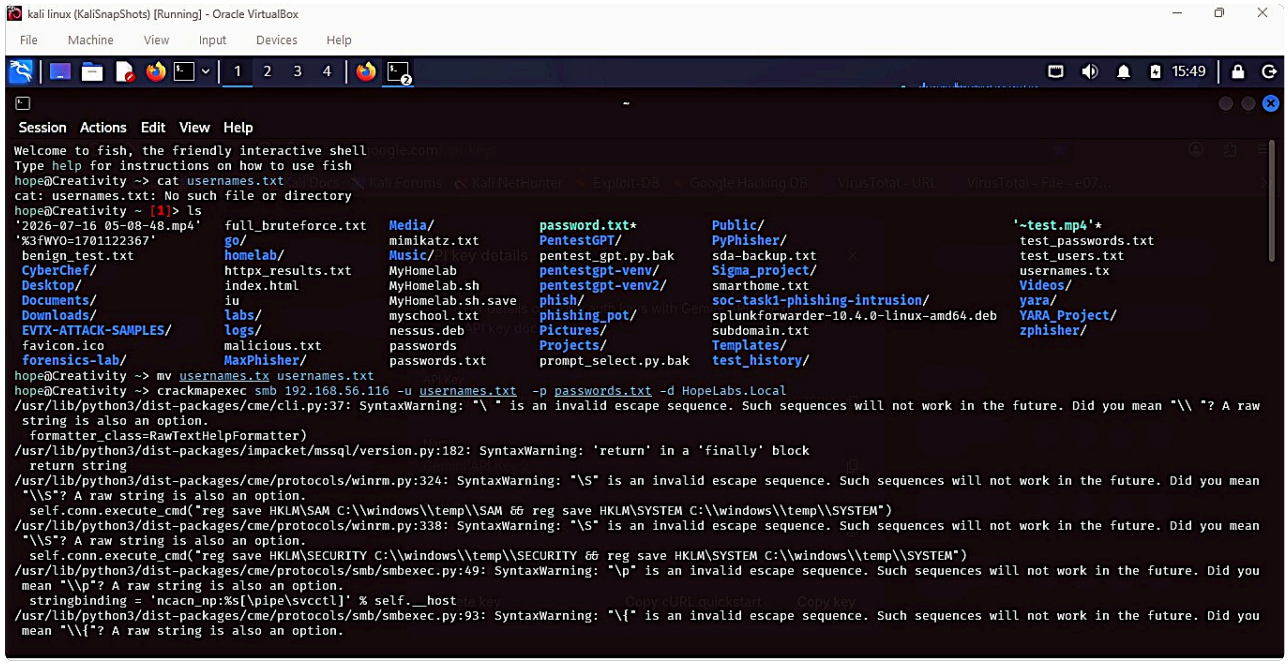


Figure 2. Kali terminal: wordlists, housekeeping and the CrackMapExec launch

Step: A **Source:** Kali desktop photo, same session continues in Figure 2b

Shows: A fish-shell session on the Kali VM ("Creativity"). `ls` shows the home directory, including `usernames.txt` / `passwords.txt` alongside unrelated project folders (CyberChef, PentestGPT, YARA_Project, a phishing-lab folder, and others left over from other work on the same box). The user renames `usernames.tx` to `usernames.txt`, then runs `crackmapexec smb 192.168.56.116 -u usernames.txt -p passwords.txt -d HopeLabs.Local`.
Demonstrates: Confirms the exact command used to launch the attack, and that it targets the Windows 10 client (192.168.56.116) in the HopeLabs.Local domain.
Limitations: On start, Python/impacket/CrackMapExec print a block of `SyntaxWarning: invalid escape sequence` messages (for `\S`, `\p`, `\{` and similar inside third-party library code). These are cosmetic warnings from the installed library version, not attack failures; they do not appear again once the tool is running.

6.2 Phase B: Verifying the Ubuntu SIEM host (VM1)

Goal: confirm both SIEMs are installed, running and reachable before generating attack traffic. **Checkpoint:** both services report active/running and both web UIs load in a browser.

- Checked the Ubuntu IP (`ip addr` → 192.168.56.102) and OS version (`lsb_release -a`).
- Checked Wazuh services: `sudo systemctl status wazuh-manager` / `wazuh-indexer`.
- Checked Splunk: `sudo systemctl status splunk`; confirmed the install at `/opt/splunk/bin/splunk`.
- Verified network connectivity to the Windows Server and Windows client (`ping -c 4`).
- Started Wazuh and Splunk, and confirmed both reported active status.
- Opened Splunk (`http://192.168.56.102:8000`) and the Wazuh dashboard (`https://192.168.56.102:443`) from Firefox.

6.3 Phase C: Configuring the Windows 10 agent (VM2)

Goal: confirm the Wazuh agent on the target can reach the manager over its two required ports before trusting any alert gap as a real negative. **Checkpoint:** both `Test-NetConnection` checks return `TcpTestSucceeded: True`, and the agent service is running.

- Opened PowerShell as Administrator and checked the Windows IP (`ipconfig`).
- Tested connectivity to Wazuh (`ping 192.168.56.102`) and verified the agent communication port (1514) and enrollment port (1515) with `Test-NetConnection` — both returned `TcpTestSucceeded: True`.
- Checked and started the Wazuh agent service: `Get-Service WazuhSvc` / `Start-Service WazuhSvc`.

6.4 Phase D: Installing Sigma on Kali

Goal: get a working, isolated Sigma CLI with the backend plugins this lab needs, before writing any rule. **Checkpoint:** `sigma plugin list` shows the Splunk, Wazuh, OpenSearch and Sysmon plugins installed, and `sigma --version` succeeds.

- Created a project directory (`mkdir -p Sigma_project/rules`) and installed prerequisites (`git`, `python3`, `python3-venv`, `pipx`).
- Installed the Sigma CLI in an isolated environment with `pipx` (`pipx ensurepath; pipx install sigma-cli`) and verified with `sigma --version`.
- Installed the Splunk, Wazuh, OpenSearch and Sysmon Sigma plugins and verified with `sigma plugin list`.
- Optionally cloned the official SigmaHQ rule repository to `~/sigma-official` for reference, kept separate from the working rules directory so the clone would not overwrite the lab's own file.

6.5 Phase E: Running the brute-force attack

Goal: generate real Event ID 4625/4624 telemetry on the target by attacking the live domain account. **Checkpoint:** the tool's output moves from a connection-level error to a Windows authentication-level error, and eventually to a logged-in confirmation.

```
crackmapexec smb 192.168.56.116 -u usernames.txt -p passwords.txt -d HopeLabs.Local
```

Table 6. CrackMapExec command anatomy

Part	Meaning
`crackmapexec smb`	Run CrackMapExec's SMB module — authenticate against the target's SMB/NTLM service, the same path Windows file sharing and domain logons use.
`192.168.56.116`	The target: the Windows 10 client, DESKTOP-T6T1N2V.
`-u usernames.txt -p passwords.txt`	Try every username against every password in the two wordlists (a full dictionary cross-product, not a single targeted guess).
`-d HopeLabs.Local`	Authenticate against this Active Directory domain rather than the local machine account database.

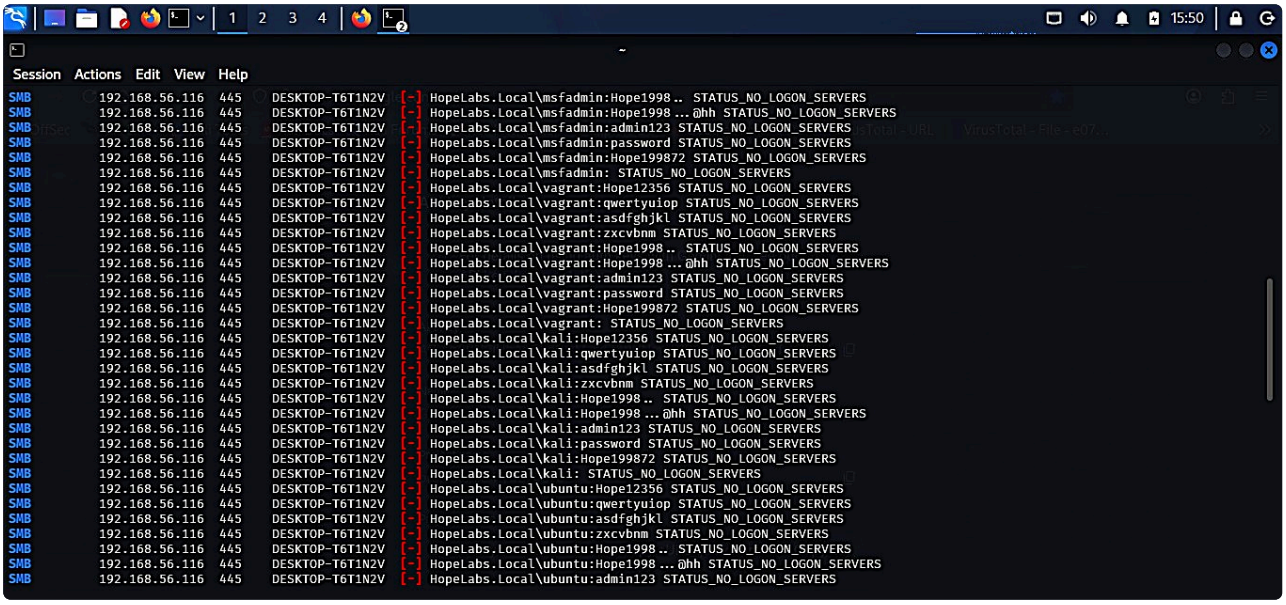


Figure 3. Early attack output: non-existent accounts rejected at the connection level

Step: E **Source:** Reused from the lab's original drop-path screenshot set; timestamp not independently legible

Shows: A CrackMapExec SMB run against 192.168.56.116, cycling through the accounts `msadmin`, `vagrant`, `kali` and `ubuntu` with a short password list for each, all returning `STATUS_NO_LOGON_SERVERS`.

Demonstrates: None of these four usernames exist as domain accounts, so the domain controller rejects the request before it becomes a real authentication attempt; this is the SMB/NTLM-layer error for an unreachable or unrecognised logon path, not a Windows 4625 audit event.

Limitations: The screen is a phone photo of the Kali desktop; line wrapping and a slightly oblique angle are visible but the text is legible.

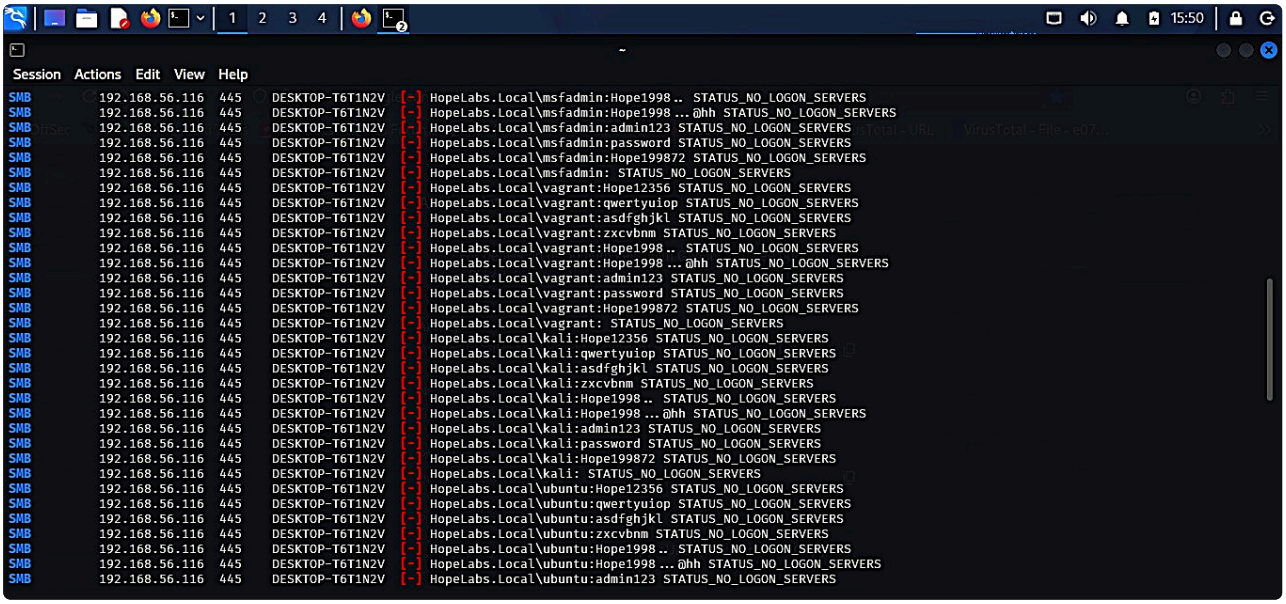


Figure 4. Continued cycling through non-existent accounts

Step: E **Source:** Kali desktop photo

Shows: The same run continuing through `vagrant`, `kali` and `ubuntu` with the remaining password candidates, still `STATUS_NO_LOGON_SERVERS` throughout.
Demonstrates: Confirms the pattern in Figure 3 is consistent across multiple non-existent usernames, not a one-off.
Limitations: Same capture conditions as Figure 3; the two images overlap in the accounts shown, which is expected given the tool retries each password per username in sequence.

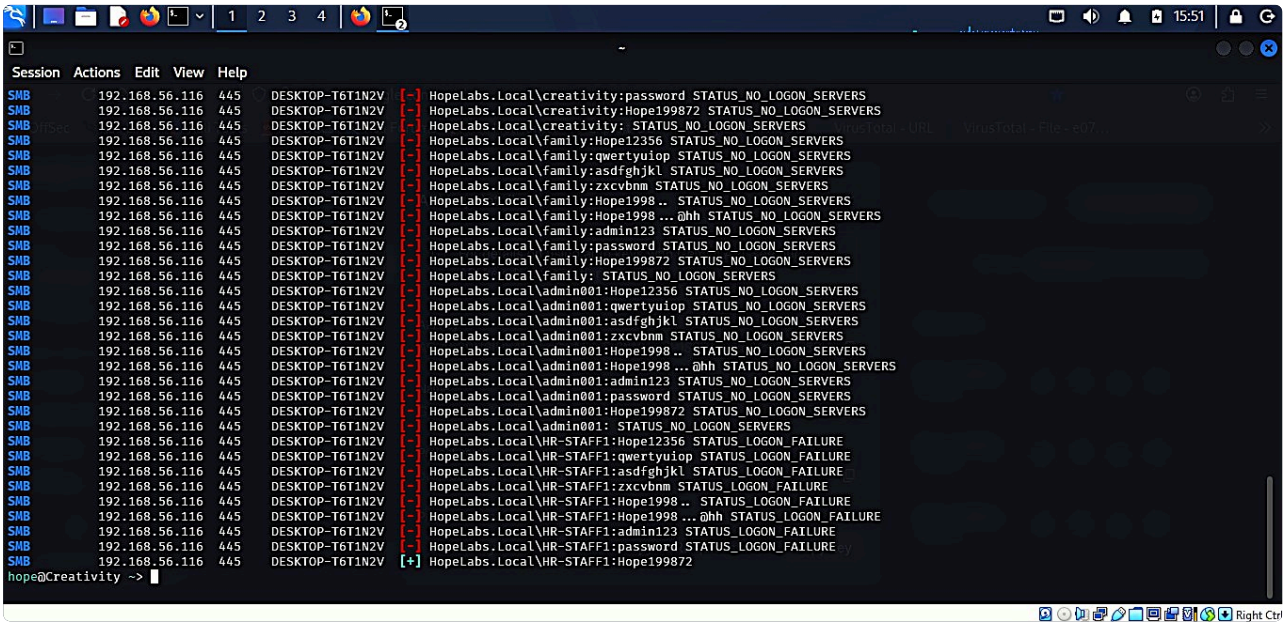


Figure 5. Attack run reaching the end of the non-existent-account list

Step: E **Source:** Kali desktop photo

Shows: The terminal scrolled further: `creativity`, `family` and `admin001` also return `STATUS_NO_LOGON_SERVERS`, and the shell returns to the `hope@Creativity` prompt at the bottom, meaning this segment of the run has completed (or was interrupted to capture the screenshot).
Demonstrates: Shows the full set of guessed local/service-sounding usernames tried before the run reaches a real domain account.
Limitations: The prompt returning does not by itself prove the whole wordlist was exhausted; it may mark a pause in the capture rather than the end of the job.

6.6 Phase F: Writing the Sigma detection rule

Goal: author a vendor-neutral detection in Sigma, covering both the single failed logon and the correlated brute-force pattern. **Checkpoint:** the YAML parses and `cat` s back cleanly, with both rule documents present in one file.

Authored two Sigma rules in `Brute_Force_SMB.yml` : a single-event detector (`win_failed_logon` , Event ID 4625) and a correlation rule (5+ failed logons for the same `TargetUserName` within a two-minute span, escalated to level `high`).

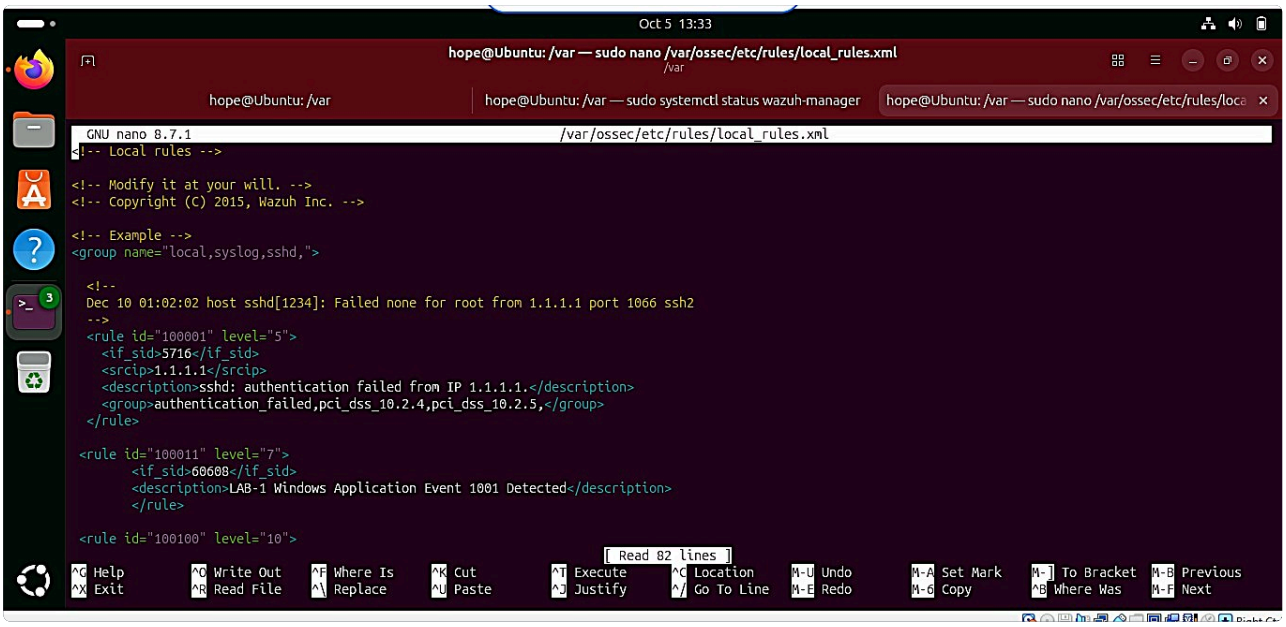


Figure 6. Wazuh local_rules.xml open in nano: the built-in rules above the custom ones

Step: F/G **Source:** Ubuntu terminal, nano 8.7.1

Shows: The top of `/var/ossec/etc/rules/local_rules.xml` in GNU nano: the file's standard header comments, the example `sshd` rule (100001) that ships with a fresh install, rule 100011 (a lab-specific Windows Application Event 1001 rule, level 7), and the start of rule 100100.
Demonstrates: Shows the custom rules were added into the standard local-rules template rather than replacing it, and that an earlier, unrelated custom rule (100011) already existed in the file from other work.
Limitations: This is a different scroll position of the same file shown again in Figures 7–8; rule 100011's purpose is not explained further in this lab's evidence.

6.7 Phase G: Deploying the Wazuh and Splunk detections

Goal: get both the Wazuh correlation rule and an equivalent Splunk SPL query actually producing alerts, each checked independently. **Checkpoint:** `wazuh-analysisd -t` prints no errors, and the Splunk search returns rows when run against the same time window.

Built the Splunk SPL query grouping `EventCode=4625` by source IP with a 5-attempt threshold (Chapter 7). Wrote and deployed the custom Wazuh rules in `/var/ossec/etc/rules/local_rules.xml` : rule 100100 (single failed Windows logon, level 10) and rule 100101 (5 matches of 100100 within 120 seconds, level 12, MITRE T1110, group `authentication_failures,brute_force`).

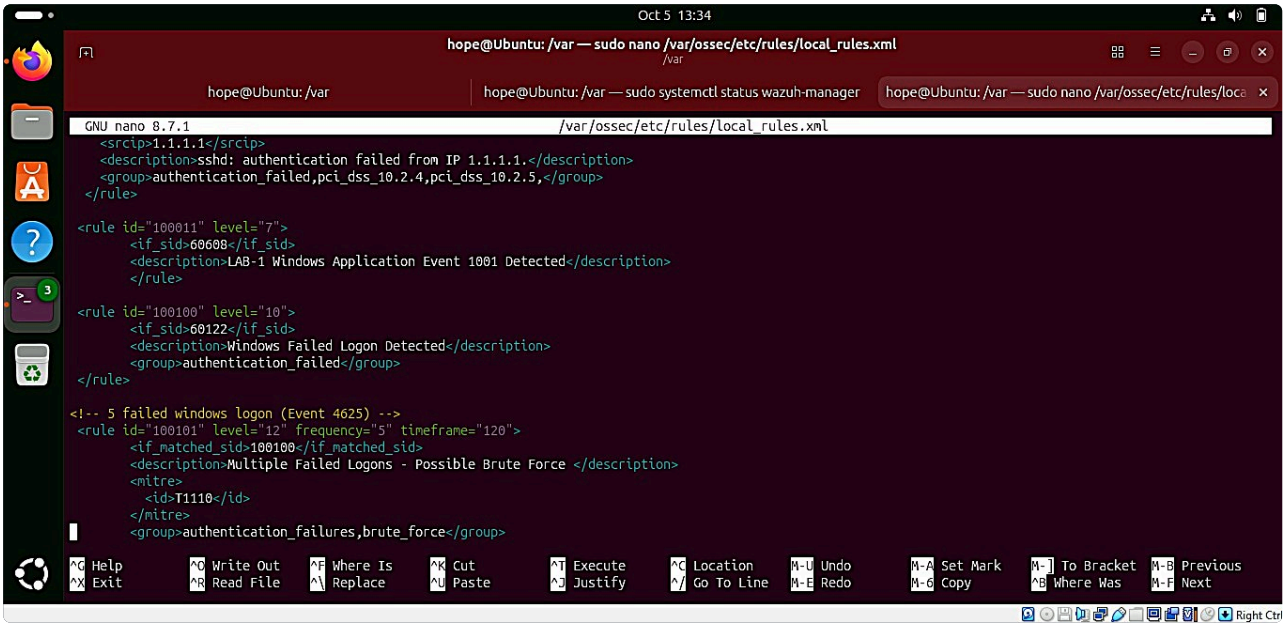


Figure 7. Rules 100100 and 100101 as deployed

Step: G **Source:** Ubuntu terminal, nano 8.7.1

Shows: Further down the same nano session: rule 100100 (`<if_sid>60122</if_sid>` , description "Windows Failed Logon Detected", group `authentication_failed`) and the start of rule 100101 (`frequency="5" timeframe="120"` , `<if_matched_sid>100100</if_matched_sid>` , description "Multiple Failed Logons - Possible Brute Force", `<mitre>` `<id>T1110</id></mitre>`).

Demonstrates: This is the exact correlation logic that produced the escalation alert: five child-rule matches, same account, two-minute window.

Limitations: The visible comment reads "5 failed windows logon (Event 4625)", confirming the intended trigger condition in the author's own words.

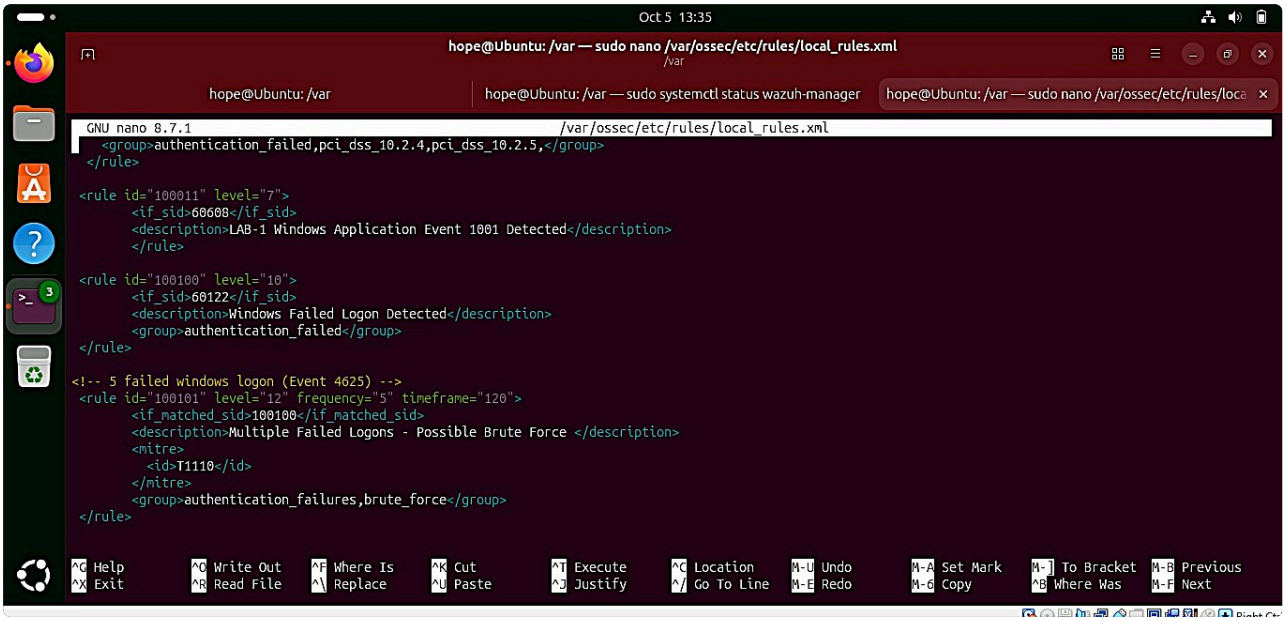


Figure 8. Rule 100101 scrolled fully into view, with its closing tags

Step: G **Source:** Ubuntu terminal, nano 8.7.1

Shows: The same rule 100101 scrolled slightly further: the full `<mitre>` block, the `<group>authentication_failures,brute_force</group>` tag, and the rule's closing `</rule>`.
Demonstrates: Confirms the rule is syntactically closed and fully tagged for ATT&CK T1110, with no missing element visible in this scroll position.
Limitations: Only the tail of the rule is visible here; the earlier lines (level, `if_matched_sid`) are confirmed in Figure 7 instead.

Validated the new rule syntax with `sudo /var/ossec/bin/wazuh-analysisd -t` (successful output, no errors reported in the working draft).

6.8 Phase H: Troubleshooting and live validation

Goal: test the Sigma conversion for both rules against the installed backend, and record the result honestly whether or not it succeeds. **Checkpoint:** the CLI's own error message, not a guess, explains any failure.

Attempted to convert the correlation rule (the 5-in-120-seconds logic) to OpenSearch. The Lucene backend rejected it: `Error: Feature required for conversion of Sigma rule is not supported by backend: Backend does not support correlation rules.` The single-event rule converted cleanly instead (to `data.win.system.eventID:4625`), confirming the limitation was backend-specific, not a mistake in how the rule was written.

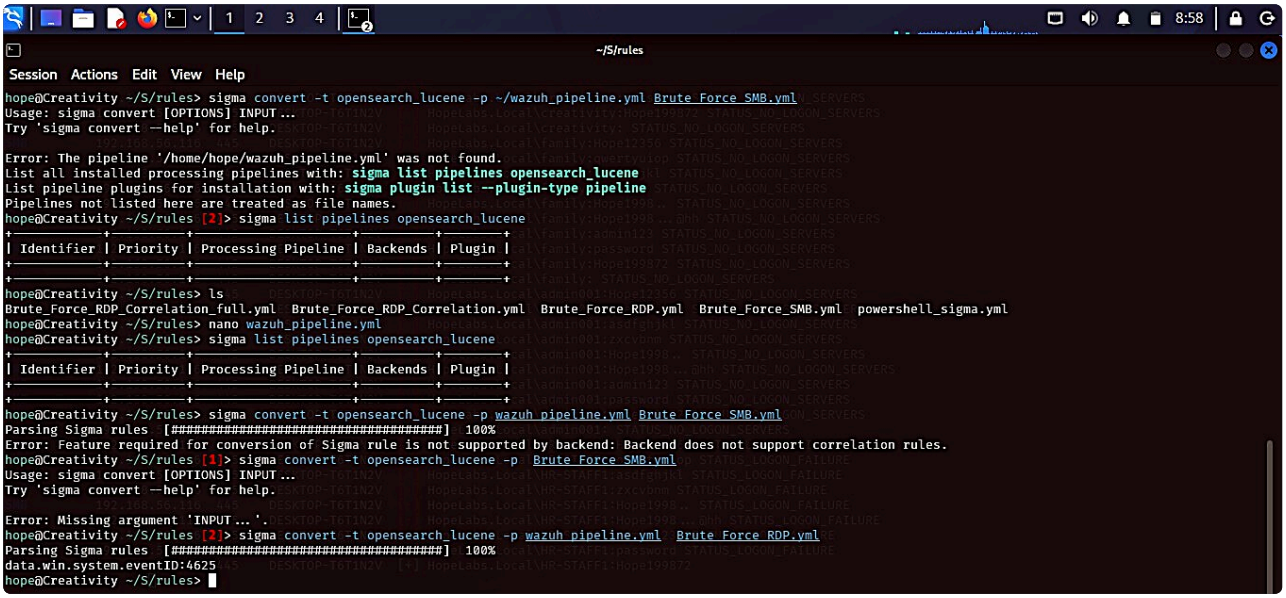


Figure 9. Sigma correlation-rule conversion failing, and the single-event rule succeeding

Step: H **Source:** Kali terminal, two sessions (13:28 and a later one at 8:58) shown together

Shows: A Kali terminal: `sigma convert -t opensearch_lucene -p ~/wazuh_pipeline.yml Brute_Force_SMB.yml` fails with "pipeline not found"; `sigma list pipelines opensearch_lucene` returns an empty table (no pipeline ships by default); after creating `wazuh_pipeline.yml`, converting `Brute_Force_SMB.yml` (which holds both rules in one file) succeeds in parsing but then fails with "Backend does not support correlation rules"; converting `Brute_Force_RDP.yml` (a single-event rule) immediately afterwards succeeds and prints `data.win.system.eventID:4625`.
Demonstrates: Shows the full diagnostic path: a missing custom pipeline first, then a genuine backend capability gap for correlation rules, isolated by testing a single-event rule from a different file against the same backend.
Limitations: The visible rule file names (`Brute_Force_RDP_Correlation_full.yml`, `Brute_Force_RDP_Correlation.yml`, `Brute_Force_RDP.yml`, `Brute_Force_SMB.yml`, `powershell_sigma.yml`) suggest more Sigma rules exist in this project than are documented in this report's evidence; only the SMB brute-force rule and the RDP single-event rule's conversion are covered here.

Confirmed, from the Wazuh Threat Hunting dashboard and event log, that rules 100100/100101 fired as designed and that the attack ultimately succeeded against `HR-STAFF1`.

CHAPTER 7

Configuration and Detection Logic

7.1 The Wazuh rules (as deployed)

```
<!-- /var/ossec/etc/rules/local_rules.xml (relevant section) -->
<rule id="100100" level="10">
  <if_sid>60122</if_sid>
  <description>Windows Failed Logon Detected</description>
  <group>authentication_failed</group>
</rule>

<!-- 5 failed windows logon (Event 4625) -->
<rule id="100101" level="12" frequency="5" timeframe="120">
  <if_matched_sid>100100</if_matched_sid>
  <description>Multiple Failed Logons - Possible Brute Force </description>
  <mitre>
    <id>T1110</id>
  </mitre>
  <group>authentication_failures,brute_force</group>
</rule>
```

7.2 Line-by-line explanation

Table 7. Rule elements explained

Element	What it does
`<rule id level>`	Unique custom ID (100000+) and severity. 100100 is level 10 (a single failure); 100101 is level 12 (the correlated brute-force alert).
`<if_sid>60122</if_sid>` (100100)	Attaches rule 100100 as a child of Wazuh's built-in Windows-logon-failure rule chain, so it is evaluated for that event type.
`frequency="5" timeframe="120"` (100101)	Correlation: fires once the referenced rule has matched 5 times within a rolling 120-second window.
`<if_matched_sid>100100</if_matched_sid>`	Names the single rule (100100) whose matches rule 100101 counts. Like the phishing lab's rule 100107, this element accepts one rule ID, not a list.
`<mitre><id>T1110</id></mitre>`	Tags the alert with the ATT&CK Brute Force technique.
`<group>`	Free-text tags used for dashboard filtering and (in rule 100100's case) the `authentication_failed` grouping already used by the built-in rule chain.

7.3 Rule inventory and status

Table 8. Custom and relevant built-in rules

Rule	Level	Detects	Parent / counted rule	Status
100100	10	A single Windows failed logon (Event ID 4625)	if_sid 60122	**Verified** (118 alerts in the 24 h window, per the dashboard)

Rule	Level	Detects	Parent / counted rule	Status
100101	12	Five 100100 matches for the same account within 120 s	if_matched_sid 100100	**Verified**: escalation observed, followed by a successful-logon alert for the same account
92652 (built-in)	6	Successful Remote Logon Detected (NTLM)	built-in	**Observed** firing for HR-STAFF1 immediately after the escalation

7.4 The Splunk SPL query

```
index=* EventCode=4625
| rex "(?i)Source Network Address.*?(?<src_ip>\S+)"
| stats count as failed_attempts by src_ip
| where failed_attempts >= 5
| sort -failed_attempts

# Alternative (counts only, no threshold filter):
index=* EventCode=4625
| stats count
```

Table 9. SPL query anatomy

Part	Meaning
`index=* EventCode=4625`	All failed-logon events, across every index, so the query does not depend on a specific index name being correct.
`| rex "(?i)Source Network Address.*?(?<src_ip>\S+)"`	Extract the source IP address from the raw Windows Security event text into a new field `src_ip`, case-insensitively.
`| stats count as failed_attempts by src_ip`	Count failed attempts per source IP.
`| where failed_attempts >= 5`	Keep only source IPs with 5 or more failures — the same threshold as Wazuh rule 100101, but counted by source IP rather than by target account.

Evidence gap. The Wazuh rule and the Splunk query use **different grouping keys** by design as written here: Wazuh rule 100101 counts failures against the same *target account* (`TargetUserName` , per the Sigma rule's `group-by`); the Splunk query counts failures from the same *source IP*. Both are legitimate brute-force signals and both fired on this attack because it was one attacker from one IP against one account, but they are not strictly equivalent queries — a distributed attack (many source IPs, one account) would be caught by the Wazuh-style grouping and missed by the Splunk query as written, and a password-spray (one IP, many accounts) would be caught by the Splunk-style grouping and missed by a strict per-account Wazuh rule. This distinction is not discussed in the author's working draft and is added here because it affects how portable the "equivalent" label really is.

7.5 The Sigma rules

```
title: Windows Failed Logon
id: 7c1f2a4e-3b8d-4e6a-9f10-2d5b8c3a1e01
name: win_failed_logon
status: experimental
description: Single failed Windows logon (Event ID 4625)
author: hope
date: 2026-10-05
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4625
  condition: selection
level: low
tags:
  - attack.credential_access
  - attack.t1110
---
title: Multiple Windows Failed Logons - Possible Brute Force
id: 9a3d5b7c-1e2f-4a6b-8c4d-5e7f9a1b2c02
status: experimental
description: 5 or more failed Windows logons for the same account within 2 minutes
author: hope
date: 2026-10-05
correlation:
  type: event_count
  rules:
    - win_failed_logon
  group-by:
    - TargetUserName
  timespan: 2m
  condition:
    gte: 5
level: high
tags:
  - attack.credential_access
  - attack.t1110
```

> **Note.** Converting the single-event rule above to the OpenSearch/Lucene backend used in this lab produces `data.win.system.eventID:4625`. The correlation rule (second document) does not convert against this backend; see Chapter 10 for the exact error and what it does and does not mean.

CHAPTER 8

Testing and Results

8.1 Test matrix

Table 10. Test scenarios and verification status

Test	Command / action	Expected	Observed	Status
T1 Non-existent accounts rejected	CrackMapExec against `msadmin`, `vagrant`, `kali`, `ubuntu`, etc.	SMB-level rejection, not a Windows 4625 audit event	`STATUS_NO_LOGON_SERVERS` for every attempt (Figures 3, 4 and 5)	**Pass** (expected behaviour for unknown accounts)
T2 Real account, wrong password	CrackMapExec reaches `HR-STAFF1` with incorrect passwords	`STATUS_LOGON_FAILURE`, and a Windows 4625 event per attempt	Observed in the terminal output (Figure 11)	**Pass**
T3 Single-failure rule	Rule 100100 on each 4625	Alert at level 10 per failure	118 authentication-failure alerts recorded for the window (dashboard tile, Figure 13)	**Pass**
T4 Correlation rule	Rule 100101: 5× 100100 within 120 s, same account	Level-12 escalation alert	"Multiple Failed Logons - Possible Brute Force" appears in the event log (Figure 10); escalation confirmed by the author's working draft	**Pass**, though not independently re-counted from `alerts.json` in this rebuild (Chapter 16)
T5 Account compromise	CrackMapExec reaches the correct password	A logged `[+]` success line and a Windows successful-logon event	`[+] HopeLabs.Local\HR-STAFF1:Hope199872` (Figure 11); built-in rule 92652 fires for the same account	**Pass**
T6 Splunk SPL query	`index=* EventCode=4625 | stats count as failed_attempts by src_ip | where failed_attempts>=5`	Rows for any source IP with 5+ failures	Query written and run per the author's working draft; its result rows were not among the images supplied for this rebuild	**Reported, not independently verified here** (Chapter 16)
T7 Sigma single-event conversion	`sigma convert -t opensearch_lucene -p wazuh_pipeline.yml Brute_Force_RDP.yml`	A valid OpenSearch query string	`data.win.system.eventID:4625` produced (Figure 9)	**Pass**

Test	Command / action	Expected	Observed	Status
T8 Sigma correlation-rule conversion	`sigma convert -t opensearch_lucene -p wazuh_pipeline.yml Brute_Force_SMB.yml`	A valid OpenSearch query for both rules	`Error: ... Backend does not support correlation rules.`	**Fail** — backend limitation, documented in Chapter 10, not fixed in this lab

8.2 What succeeded, what failed, what remains unverified

- **Succeeded:** the attack reaching and compromising the target account; both Wazuh rules firing as designed; the Sigma single-event rule converting cleanly; the built-in successful-logon rule (92652) corroborating the compromise.
- **Failed, and reported as a finding rather than hidden:** the Sigma correlation rule did not convert against this lab's OpenSearch/Lucene backend.
- **Not independently re-verified in this rebuild:** the exact alert counts (22,069 / 21,898 / 118 / 2) and the "667 pages / ~10,000 events" figure are taken from the dashboard screenshots and the author's own working draft, not re-derived from `alerts.json` with `grep` / `uniq -c` the way the companion phishing-detection lab's counts were. The Splunk query's actual result rows, the Splunk dashboard panel, and a raw Splunk event detail described in the working draft were not supplied as images for this report and so are not reproduced as figures here (see Chapter 16).

CHAPTER 9

Evidence and Screenshot Analysis

The figures below appear in the order the procedure in Chapter 6 introduces them. This chapter adds the Wazuh dashboard evidence that supports the result totals quoted in Chapters 1 and 8.

9.1 Image preparation and evidence integrity

- Original photographs and screenshots are retained unchanged.
- Report copies are cropped to the relevant screen area, contrast-adjusted and sharpened. No text, value, timestamp, rule ID, command or error message was edited, redrawn, added or removed.
- No annotations were added, so every pixel shown is original evidence.
- All figures in this chapter are phone or screen photographs of a VirtualBox guest window; several show perspective distortion, reflections or a visible taskbar/host desktop around the guest window, which is left in rather than cropped away when it does not obscure evidence.
- Two pieces of evidence referenced in the author's own working draft of this report (a Splunk dashboard pie-chart panel, and a raw Splunk event-detail page showing the 4625 failure reason and sub-status codes) were not among the images supplied for this rebuild. They are described from the working draft's text in Chapters 1, 8 and 16, but are not presented as figures here, consistent with the rule against reconstructing evidence that was not actually supplied.

9.2 Wazuh dashboard evidence

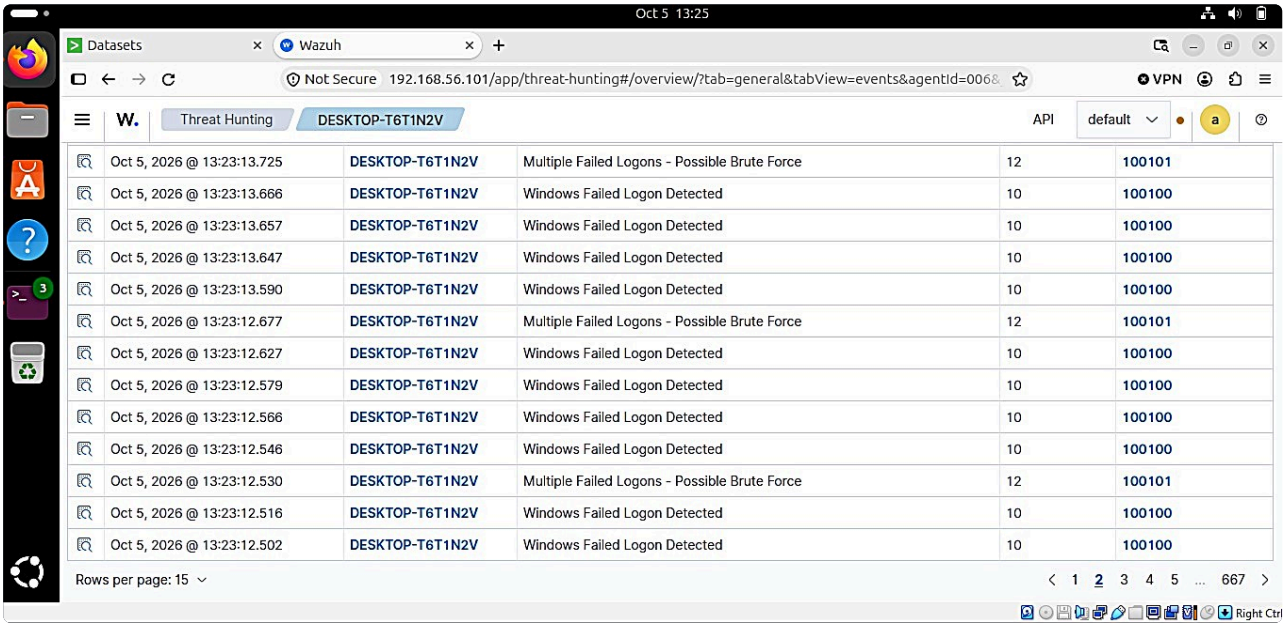


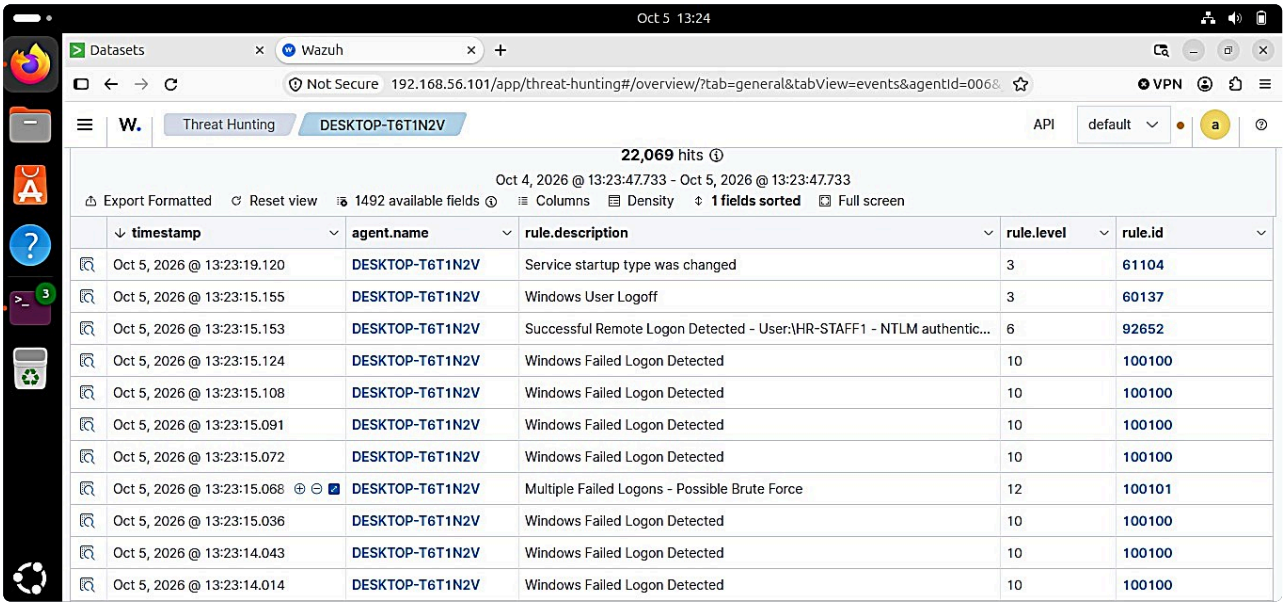
Figure 10. Event log: the escalation pattern, page 2 of 667

Step: G/H **Source:** Wazuh dashboard, Oct 5, 13:25

Shows: Threat Hunting events for DESKTOP-T6T1N2V, rows per page 15, page 2 of 667. Visible rows alternate "Windows Failed Logon Detected" (level 10, rule 100100) with "Multiple Failed Logons - Possible Brute Force" (level 12, rule 100101) at 13:23:12–13:23:13 on Oct 5, 2026.

Demonstrates: Shows rule 100101 actually escalating inside the real event stream, interleaved with the rule 100100 events it counts — not just defined in the rules file, but firing.

Limitations: This is one page of 667; the total of 667 pages at 15 rows/page is roughly 10,000 events for this agent in the queried window, which the author's draft states but which this rebuild did not independently recompute from the raw file.



timestamp	agent.name	rule.description	rule.level	rule.id
Oct 5, 2026 @ 13:23:19.120	DESKTOP-T6T1N2V	Service startup type was changed	3	61104
Oct 5, 2026 @ 13:23:15.155	DESKTOP-T6T1N2V	Windows User Logoff	3	60137
Oct 5, 2026 @ 13:23:15.153	DESKTOP-T6T1N2V	Successful Remote Logon Detected - User:\HR-STAFF1 - NTLM authentic...	6	92652
Oct 5, 2026 @ 13:23:15.124	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100
Oct 5, 2026 @ 13:23:15.108	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100
Oct 5, 2026 @ 13:23:15.091	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100
Oct 5, 2026 @ 13:23:15.072	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100
Oct 5, 2026 @ 13:23:15.068	DESKTOP-T6T1N2V	Multiple Failed Logons - Possible Brute Force	12	100101
Oct 5, 2026 @ 13:23:15.036	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100
Oct 5, 2026 @ 13:23:14.043	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100
Oct 5, 2026 @ 13:23:14.014	DESKTOP-T6T1N2V	Windows Failed Logon Detected	10	100100

Figure 11. Event log: earliest visible page, with the 4625/4624 context

Step: G/H **Source:** Wazuh dashboard, Oct 5, 13:24

Shows: A later/different scroll of the same Threat Hunting events view: rows for "Service startup type was changed" (rule 61104), "Windows User Logoff" (rule 60137), "Successful Remote Logon Detected - User:\HR-STAFF1 - NTLM authenticat..." (rule 92652, level 6), followed by a run of "Windows Failed Logon Detected" (rule 100100) and one "Multiple Failed Logons - Possible Brute Force" (rule 100101, level 12).
Demonstrates: This is the clearest single screenshot of the full signature described in Chapters 1 and 10: failures escalating to a brute-force alert, with a successful NTLM logon for HR-STAFF1 appearing in the same short window — the pattern the author's conclusion calls "the textbook signature of a successful brute force."
Limitations: The successful-logon row appears above the failed-logon run in this particular scroll position, i.e. chronologically after it in a timestamp-descending list; the exact sub-second ordering of "last failure → success" is not fully legible in this single screenshot.

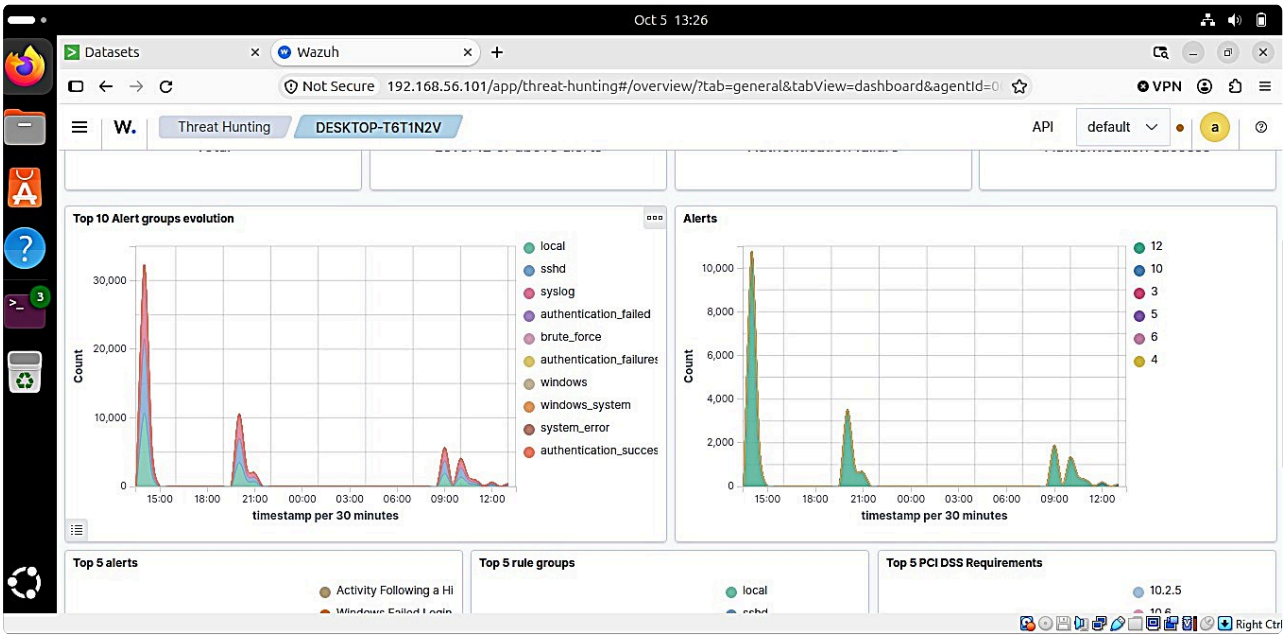


Figure 12. Dashboard overview: alert-group evolution and alert-level timeline

Step: G/H **Source:** Wazuh dashboard, Oct 5, 13:26

Shows: Two charts for agent DESKTOP-T6T1N2V over roughly 24 hours (14:00 to 12:00+): "Top 10 Alert groups evolution" (local, sshd, syslog, authentication_failed, brute_force, authentication_failures, windows, windows_system, system_error) and "Alerts" by level (12, 10, 3, 5, 6, 4), both showing the same three activity bursts around 15:00, 21:00 and the 09:00–12:00 period.
Demonstrates: The brute-force and authentication_failed/authentication_failures groups rise together with the overall alert volume in the same bursts, consistent with the attack driving the spike rather than unrelated noise.
Limitations: The x-axis spans roughly 24 hours but the exact start/end timestamps are small and not fully legible in this capture; the three bursts are not individually dated to specific lab sessions (Oct 5 vs Oct 7 vs Oct 14) from this image alone.

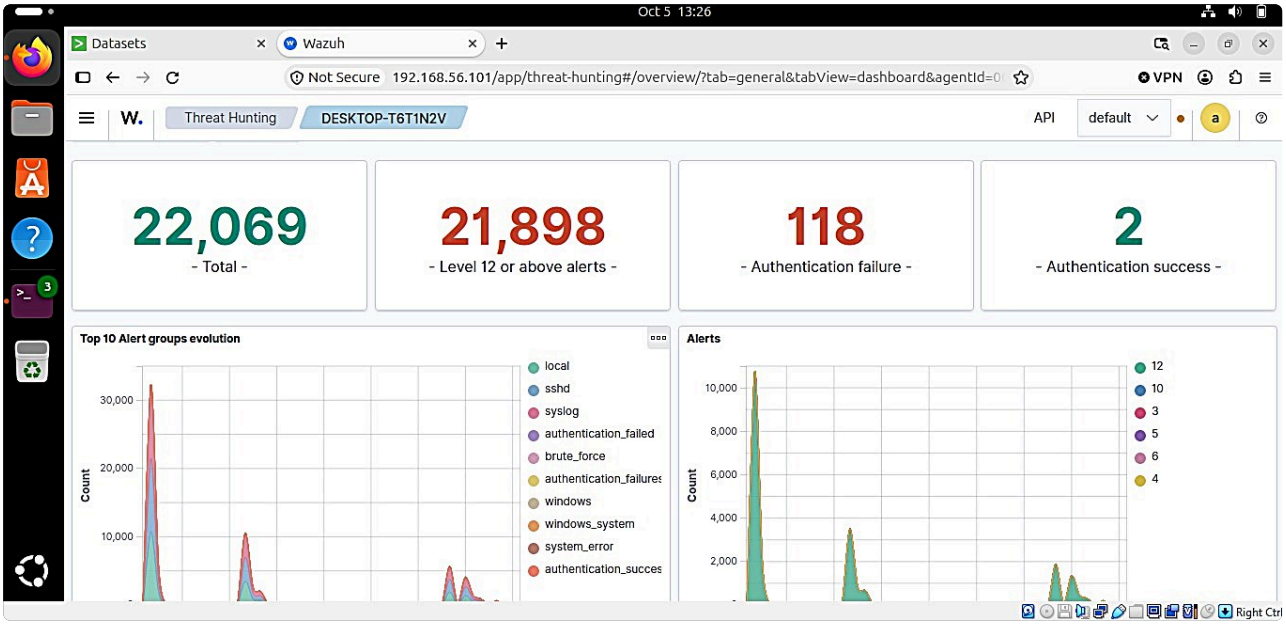


Figure 13. Dashboard summary tiles: the headline counts

Step: G/H **Source:** Wazuh dashboard, Oct 5, 13:26

Shows: Four stat tiles for agent DESKTOP-T6T1N2V: **22,069** Total, **21,898** Level 12 or above alerts, **118** Authentication failure, **2** Authentication success — directly below the same alert-group evolution chart as Figure 12.

Demonstrates: This is the primary source for the headline numbers quoted in the Executive Summary: 118 failed-logon-driven alerts and 2 successful logons recorded for this agent in the dashboard's selected window.

Limitations: These are dashboard-computed aggregates over whatever time range was selected in the UI at capture time; the exact range is given on a separate part of the same page (Figure 11, "Oct 4, 2026 @ 13:23:47.733 – Oct 5, 2026 @ 13:23:47.733", i.e. a 24-hour window), and the figure was not independently cross-checked against `grep -c` on `alerts.json`.

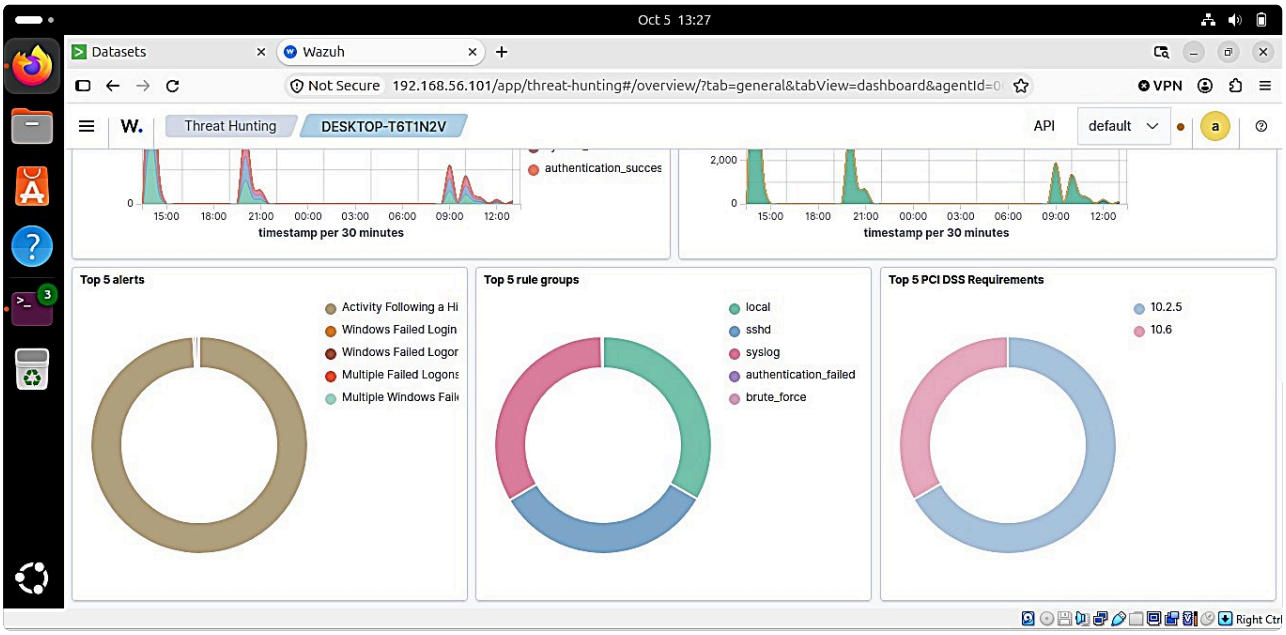


Figure 14. Top 5 alerts, rule groups and PCI DSS requirements

Step: G/H **Source:** Wazuh dashboard, Oct 5, 13:27

Shows: Three donut charts: Top 5 alerts (Activity Following a High-Volume..., Windows Failed Logon, Windows Failed Logon..., Multiple Failed Logons, Multiple Windows Fail...); Top 5 rule groups (local, sshd, syslog, authentication_failed, brute_force); Top 5 PCI DSS Requirements (10.2.5, 10.6).

Demonstrates: Confirms the `brute_force` and `authentication_failed` groups tagged on rules 100100/100101 are among the dominant rule groups for this agent in the window, and that Wazuh's own PCI DSS mapping classifies this activity under requirements 10.2.5 and 10.6 (both concerned with logging and monitoring access to audit trails/system components).

Limitations: Donut charts show relative proportion, not absolute counts; the PCI DSS tags are Wazuh's standard rule-group mapping and are not a claim that this lab was assessed against PCI DSS.

CHAPTER 10

Troubleshooting and Root-Cause Analysis

10.1 Issue 1: early attack attempts rejected at the SMB layer

Table 11. Issue 1 analysis

Item	Detail
Symptom	The first several minutes of the CrackMapExec run returned `STATUS_NO_LOGON_SERVERS` for every username/password pair tried (Figures 3, 4 and 5), with no corresponding rise in Wazuh alerts.
Possible causes considered	(a) network/firewall issue blocking SMB; (b) the domain controller unreachable; (c) the usernames tried simply do not exist as domain accounts.
Diagnosis	The same symptom persisted across many different usernames (`msadmin` , `vagrant` , `kali` , `ubuntu` , `creativity` , `family` , `admin001`) with no connection-level errors reported elsewhere in the session, and the attack later succeeded against a real account (`HR-STAFF1`) using the same tool and path — ruling out (a) and (b).
Conclusion	(c): these are plausible local/service account names that do not correspond to actual HopeLabs.Local domain users, so the domain controller returns a generic no-logon-server-style rejection rather than processing them as real authentication attempts. This is expected tool behaviour against a wordlist of generic usernames, not a lab fault.

10.2 Issue 2: Sigma correlation rule does not convert to the OpenSearch backend

Table 12. Issue 2 analysis

Item	Detail
Symptom	`sigma convert -t opensearch_lucene -p wazuh_pipeline.yml Brute_Force_SMB.yml` parses the file (100% parsing reported) but then fails: `Error: Feature required for conversion of Sigma rule is not supported by backend: Backend does not support correlation rules.` (Figure 9).
Possible causes considered	(a) a mistake in the correlation rule's YAML syntax; (b) a missing or wrong field-mapping pipeline; (c) the installed backend plugin genuinely lacking correlation support.
Diagnostic step	A separate, single-event Sigma rule (`Brute_Force_RDP.yml`) was converted against the exact same backend and pipeline immediately afterwards, and it succeeded, producing `data.win.system.eventID:4625` .
Conclusion	(c): the backend itself, not the rule, is the limiting factor. The correlation YAML is syntactically valid Sigma (it parses), and the pipeline and backend both work for single-event rules against the same logsource, so the failure isolates cleanly to "this OpenSearch/Lucene backend plugin, as installed, does not implement Sigma's correlation feature" — confirmed by the CLI's own error text, not inferred.

10.3 Issue 3: two different IP addresses recorded for the Ubuntu SIEM host

The author's working draft gives VM1's IP as 192.168.56.102 in the infrastructure table and the install/connectivity steps, but as 192.168.56.101 when describing the Wazuh dashboard URL and the live dashboard screenshots' browser address bar (visible as `192.168.56.101/app/threat-hunting...` in Figures 11–14). Both addresses are

used consistently within their own contexts (102 for Splunk and agent enrollment, 101 for the Wazuh web UI), which suggests either two IPs on the same host (for example, a second host-only adapter or alias) or a naming inconsistency between the agent-facing and browser-facing addresses, rather than two different machines. This report carries the discrepancy through as recorded, consistent with the instruction not to silently resolve a conflict between sources without noting it.

10.4 Issue 4: duplicate/overlapping screenshots

Figures 3, 4 and 5 all show the same CrackMapExec run cycling through non-existent usernames, with overlapping account names visible across more than one image. They are kept as separate figures rather than merged into one, because together they establish that the pattern held consistently across the full set of generic usernames tried, which a single cropped image would not show as clearly; no new technical information is claimed from the overlap itself.

CHAPTER 11

Security Findings and Risk Analysis

Table 13. Findings and confidence

#	Finding	Type	Evidence	Likely impact
F1	The domain account HR-STAFF1 was compromised by a dictionary attack within the lab's test window, using a password present in a common wordlist.	**Confirmed** (simulated)	Figure 11, the `[+]` success line, built-in rule 92652	A weak or wordlist-guessable password is sufficient for an attacker to gain domain credentials, independent of any detection capability.
F2	Both failed-logon volume and the correlated brute-force escalation were detected and alerted on by Wazuh before/around the moment of compromise.	**Confirmed**	Figures 10, 11 and 13; rules 100100/100101	Detection worked as designed; the gap is between detection and automatic response (Finding F4).
F3	The organisation's chosen detection-portability tool (Sigma) cannot express this lab's correlation logic against the installed OpenSearch/Lucene backend.	**Confirmed**	Figure 9, the CLI's own error text	A team relying on Sigma to move multi-event detections like this one between SIEMs would silently lose the correlation stage unless they verify backend support first; the single-event stage alone is not sufficient to catch a brute force without the threshold.
F4	No automated response (lockout, block, disable) was configured to act on the rule 100101 alert; the attack was allowed to continue to a successful logon.	**Confirmed** (by design — this lab's scope was detection, not response)	Attack narrative in Chapter 6; no lockout evidence in the screenshots	In a non-lab setting, a detection with no response still leaves a window for the attacker to reach a working password.
F5	The Wazuh rule groups failures by target account; the Splunk query groups failures by source IP. The two detections, while both effective here, are not strictly equivalent.	Design observation (not a defect)	Chapter 7 rule and query text	A distributed brute force (many IPs, one account) or a password spray (one IP, many accounts) could be caught by one detection and missed by the other, depending on which grouping key is used.
F6	22,069 total alerts and 118 authentication-failure alerts for one agent in 24 hours is a high volume for a human analyst to triage manually.	Observation	Figure 13	At this volume, correlation rules like 100101 are doing real work reducing noise to a small number of actionable escalations; without them, an analyst would face thousands of low-level rows.

CHAPTER 12

Mitigation and Remediation

Items M1–M3 reflect what this lab actually built and verified. Everything else is a recommendation that was not implemented here.

Table 14. Mitigations

#	Recommendation	Objective and approach	Benefit / impact	Verify	Status
M1	Deploy the Wazuh correlation rule 100101 (frequency 5, timeframe 120) on top of the single-event rule 100100	Escalate repeated failures against one account into one actionable alert	Reduces 100+ low-level rows to a handful of high-severity ones	Dashboard/event-log entries for rule 100101 (Figure 10, 11)	**Implemented**
M2	Build an independent Splunk SPL query over the same event type	Confirm the detection is not a single-tool artefact	Cross-platform corroboration	Query text in Chapter 7	**Implemented** (query written and reported run; result rows not independently re-verified here, Chapter 16)
M3	Author Sigma rules for the same logic and test conversion to the Wazuh/OpenSearch backend	Make the detection logic portable; find backend gaps before relying on them	Surfaced a real, documented backend limitation (Chapter 10)	Figure 9's error text	**Implemented** (partial: single-event only)
M4	Enforce a stronger password policy (length/complexity, breached-password screening) and/or MFA for HR-STAFF1 and similarly privileged accounts	Prevent the dictionary attack from succeeding at all, not only detect it	Removes the vulnerability this attack exploited	Attempt the same dictionary attack post-policy-change and confirm it no longer succeeds within a reasonable window	Recommended
M5	Add a Wazuh active-response action to automatically disable or block the account/source once rule 100101 fires	Close the gap between detection (rule 100101) and the eventual compromise observed in this lab	Stops the attack before the password is reached, instead of only alerting after	Re-run the attack and confirm the account is disabled/IP blocked before a successful logon occurs	Recommended
M6	Re-test the Sigma correlation rule against a backend/pipeline that supports Sigma correlation rules, or track the OpenSearch Lucene backend's roadmap for that feature	Restore true cross-platform portability for the full two-stage detection, not just the single-event half	Removes the single point of platform lock-in identified in Finding F3	Successful conversion of `Brute_Force_SMB.yml`'s correlation document	Recommended

#	Recommendation	Objective and approach	Benefit / impact	Verify	Status
M7	Expand the Splunk dashboard beyond a single panel: add a time-series trend and a scheduled alert that fires once a source IP crosses the 5-failure threshold	Turn the ad-hoc SPL query into a standing detection, matching Wazuh's always-on rule	Splunk side reaches operational parity with the Wazuh side	A scheduled search's Triggered Alerts list shows an entry for this attack	Recommended
M8	Decide and document a single grouping key (account vs source IP) — or run both — for the brute-force detection across tools	Close the gap identified in Finding F5 between the two "equivalent" detections	Avoids a false sense of parity between tools that actually catch different attack shapes	Test both a distributed-IP attack and a password-spray attack and confirm both detections fire appropriately	Recommended

CHAPTER 13

What I Learned

This section is written in the first person and reflects the actual work in this lab.

Conceptual knowledge

I learned to read a brute-force attack as a sequence with its own internal logic, not a single event: unknown accounts rejected at the SMB/NTLM layer before any Windows audit event is even generated, then real accounts producing genuine 4625 failures, then eventually a success. Seeing `STATUS_NO_LOGON_SERVERS` turn into `STATUS_LOGON_FAILURE` once the tool reached a real account was the clearest signal in the whole exercise that the wordlist attack was behaving exactly as expected.

Practical technical knowledge

I learned to write a two-stage Wazuh correlation rule (a single-event child rule plus a `frequency` / `timeframe` / `if_matched_sid` parent) and to validate it with `wazuh-analysisd -t` before trusting it. I learned to write the matching Sigma correlation syntax (`correlation: type: event_count`, `group-by`, `timespan`, `condition: gte`) and to build an SPL query that performs the equivalent grouping and thresholding independently, in a different query language.

Troubleshooting and tool-portability limits

The most concrete lesson came from a failure, not a success: converting a working Sigma correlation rule to the Wazuh/OpenSearch backend and getting a clear, specific error instead of either success or a silent wrong answer. I learned to isolate whether that was my rule or the backend by converting a second, simpler rule against the exact same backend and pipeline — it worked, which told me the correlation feature itself, not my YAML, was unsupported. That is a useful habit beyond this one tool: when something fails, change one variable at a time until the failure moves or disappears.

Evidence analysis and detection engineering

Cross-checking the Wazuh dashboard's own numbers (118 failures, 2 successes) against the raw event-log view (Figures 10 and 11) showed me the same pattern two different ways, which is a stronger form of verification than trusting either view alone. I also learned, in writing this version of the report, that a dashboard tile is not the same kind of evidence as a `grep -c` over the raw alert file, and that I should say so plainly rather than presenting both as equally solid.

Areas for further development

- Re-derive the 22,069/21,898/118/2 figures from `alerts.json` directly, the way the companion phishing-detection lab's rule counts were verified.
- Capture the Splunk query's actual result rows and the Splunk dashboard panel as evidence, not just describe them.
- Test a backend/pipeline that does support Sigma correlation rules, to see the full two-stage detection convert.
- Implement and test an active-response action (Mitigation M5) so detection and response close the loop this lab left open.
- Test the grouping-key gap identified in Finding F5 with a distributed-IP attack and a password-spray attack.

CHAPTER 14

Practical Knowledge and Competencies

Table 15. Skills demonstrated, with the evidence for each

Area	Demonstrated by	Evidence	Level shown
SIEM administration & correlation-rule engineering	Deployed and tuned Wazuh rules 100100/100101 (threshold-based brute-force correlation, MITRE-tagged)	Figures 6, 7, 8	Independent
Threat-detection content authoring (Sigma)	Authored vendor-neutral single-event and correlation Sigma rules; converted them toward the Wazuh/OpenSearch backend	Figure 9	Independent, with a documented limitation found
SPL development	Built a Splunk query grouping and thresholding Event ID 4625 by source IP	Chapter 7 query text	Independent (query written; results not independently re-verified here)
Windows/AD security event analysis	Interpreted Event ID 4625 sub-status behaviour and NTLM logon success (rule 92652) to confirm the compromise	Figure 11	Independent
Offensive tooling for defensive validation	Ran a controlled CrackMapExec dictionary attack to generate realistic telemetry	Figures 2–5	Independent
MITRE ATT&CK mapping	Tagged the correlation rule and both Sigma rules to T1110 (Brute Force)	Chapter 7	Independent
Lab infrastructure and network design	Built and segmented a 4-VM environment (VirtualBox, NAT + host-only adapters, an AD domain)	Chapter 4	Independent
Incident investigation and reporting	Correlated Wazuh and (reported) Splunk evidence into a root-cause narrative, including an honest account of what was not independently re-verified	This report	Independent
Automated response engineering	Not attempted in this lab (see Mitigation M5)	—	**Not demonstrated**

14.1 Recommended next exercises

1. Implement a Wazuh active-response action that disables the account or blocks the source IP once rule 100101 fires, and re-run the attack to confirm it is stopped before a successful logon.
2. Re-derive the headline alert counts from `alerts.json` with `grep -c` / `uniq -c`, the way the companion phishing-detection lab's rule counts were independently verified.
3. Find or build a Sigma backend/pipeline that supports correlation rules, and convert `Brute_Force_SMB.yml`'s correlation document successfully.
4. Re-test the Splunk query and capture its actual result rows and the SOC Brute Force Detection dashboard panel as evidence.

5. Test the Wazuh-vs-Splunk grouping-key gap (Finding F5) with a distributed-source-IP attack and a password-spray attack.

CHAPTER 15

Reproduction Guide

This guide lets another learner recreate the lab. Use the checklist first, then the detailed parts that follow.

Table 16. Reproduction checklist

#	Do this	Where	Pass when	Detail
R1	Build the four-VM environment and promote VM3 to a domain controller	VirtualBox, VM3	`HopeLabs.Local` forest exists; a test account (e.g. HR-STAFF1) is created	Part 0
R2	Install Sysmon on the Windows client (optional, for enhanced telemetry) and confirm domain join	VM2	`(Get-WmiObject Win32_ComputerSystem).Domain` returns `HopeLabs.Local`	Part 0
R3	Install the Wazuh manager/indexer and Splunk Enterprise on the Ubuntu host	VM1	Both services report active/running	Part 0, Phase B
R4	Enroll the Wazuh agents on VM2 and VM3	VM1 + agents	`Test-NetConnection` to ports 1514 and 1515 both succeed from VM2	Part 0, Phase C
R5	Build the wordlists and install CrackMapExec on the attacker box	VM4	`crackmapexec --version` succeeds	Phase A
R6	Write and deploy the Wazuh rules 100100/100101	VM1	`sudo /var/ossec/bin/wazuh-analysisd -t` prints no errors	Phase G
R7	Build the Splunk SPL query	VM1, Splunk	Query runs without a syntax error against `index=* EventCode=4625`	Phase G
R8	Run the attack	VM4 → VM2	Output moves from `STATUS_NO_LOGON_SERVERS` to `STATUS_LOGON_FAILURE` to a logged `[+]` success	Phase E
R9	Confirm the detections fired	VM1, Wazuh dashboard and Splunk	Rule 100100 and 100101 entries appear in the event log; the Splunk query returns the attacking source IP with 5+ failures	Phase G/H
R10	(Optional) Author and convert the Sigma rules	VM4	The single-event rule converts; the correlation rule's error, if any, matches the one in Chapter 10	Phase D/F/H

15.1 Part 0: base infrastructure setup

Everything below is written so a reader can rebuild the environment from nothing and reproduce the same detections. Download and have ready before starting: Oracle VirtualBox (+ Extension Pack), a Windows Server 2019 evaluation ISO, a Windows 10 ISO, an Ubuntu Server 22.04 LTS ISO, a Kali Linux ISO, the SecLists wordlists, Wazuh 4.x, Splunk Enterprise/Universal Forwarder, and Sysmon with a community config.

1. Install Oracle VirtualBox and the Extension Pack on the host machine.
2. Create four VMs, each with two adapters — Adapter 1: NAT, Adapter 2: Host-only (192.168.56.0/24): VM1 Ubuntu Server 22.04 LTS (4 vCPU/8 GB/60 GB, Wazuh + indexer, Splunk); VM2 Windows 10 (2 vCPU/4

GB/60 GB, domain-joined endpoint); VM3 Windows Server 2019 (2 vCPU/4 GB/80 GB, domain controller); VM4 Kali Linux (2 vCPU/4 GB/40 GB, attacker box).

3. On VM3, install the AD DS role and promote it to a new forest.

```
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools
Install-ADDSForest -DomainName "HopeLabs.Local" -DomainNetbiosName "HOPELABS" -InstallDNS
```

1. In Active Directory Users and Computers (or via `New-ADUser`), create an OU and the test account used later in the attack:

```
New-ADUser -Name "HR-STAFF1" -SamAccountName "HR-STAFF1" \
  -AccountPassword (Read-Host -AsSecureString "Password") -Enabled $true
```

1. Join VM2 (Windows 10) to HopeLabs.Local and confirm with `(Get-WmiObject Win32_ComputerSystem).Domain`.
2. Install Sysmon on VM2 for enhanced endpoint telemetry:

```
Sysmon64.exe -accepteula -i sysmonconfig.xml
```

1. Install the Wazuh manager and indexer on VM1 using the official all-in-one installer:

```
curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

1. Install the Wazuh agent on VM2 and VM3, pointing each at the manager's host-only IP (192.168.56.102), then enroll and start the service (exact commands in Phase C).
2. Install Splunk Enterprise on VM1:

```
wget -O splunk.tgz "<Splunk Enterprise .tgz download URL>"
sudo tar xvzf splunk.tgz -C /opt
sudo /opt/splunk/bin/splunk start --accept-license
```

1. Install the Splunk Universal Forwarder on VM2, point it at the Ubuntu indexer (192.168.56.102:9997), and configure it to monitor the Windows Security event log (`WinEventLog://Security`) so Event ID 4625 events reach Splunk alongside Wazuh.

> **Note.** With the four VMs built, the domain established, and both SIEMs installed and talking to their agents/forwarders, the environment matches the one described in Chapter 4, and Phases A–H in Chapter 6 can be followed as written.

15.2 Part A: run the attack

```
crackmapexec smb 192.168.56.116 -u usernames.txt -p passwords.txt -d HopeLabs.Local
```

Expected: early output shows `STATUS_NO_LOGON_SERVERS` for non-existent usernames, then `STATUS_LOGON_FAILURE` once the tool reaches a real domain account, then a logged `[+]` line once the correct password is tried.

15.3 Part B: verify in Wazuh

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

Expect no errors. Then, in the Wazuh dashboard, open Threat Hunting for the target agent and confirm "Windows Failed Logon Detected" (rule 100100) rows and at least one "Multiple Failed Logons - Possible Brute Force" (rule 100101) row after five failures land within two minutes against the same account.

15.4 Part C: verify in Splunk

```
index=* EventCode=4625
| rex "(?i)Source Network Address.*?(?<src_ip>\S+)"
| stats count as failed_attempts by src_ip
| where failed_attempts >= 5
| sort -failed_attempts
```

Expect at least one row, for the attacking source IP, with a count of 5 or more.

15.5 Troubleshooting

Symptom	Likely cause	Fix
Every attempt returns `STATUS_NO_LOGON_SERVERS`, even for the real account	Domain not reachable, or `-d` domain name wrong/misspelled	Confirm DNS resolves `HopeLabs.Local` from VM4; confirm VM3 is up and reachable (`ping 192.168.56.115`)
Rule 100101 never fires even though many 100100 alerts appear	Failures are spread across more than 120 seconds, or against more than one account	Re-run the attack against a single known account in a tight loop; check the rule's `timeframe` value
`wazuh-analysisd -t` reports an error after editing the rules	XML syntax error, or a reference to a rule ID that does not exist yet / is defined later in the file	Check matching tags and that `if_matched_sid` points at a rule defined earlier in the file
Splunk query returns no rows	Forwarder not sending the Security log, or index name mismatch	Confirm the Universal Forwarder's `inputs.conf` monitors `WinEventLog://Security` and check the index with `index=* | stats count by index`
Sigma correlation rule fails to convert	Expected with this lab's OpenSearch/Lucene backend plugin (Chapter 10)	Convert the single-event rule instead, or use a backend/pipeline known to support Sigma correlation rules

15.6 Cleanup and rollback

1. If a lockout or response action was configured and triggered, re-enable the test account.
2. Reset `HR-STAFF1` 's password to something not in a common wordlist once testing is complete.
3. Take a VM snapshot once everything works, before any further changes.
4. Remove or archive the wordlists and attack logs if the lab environment will be reused or shared.

CHAPTER 16

Limitations and Future Improvements

- **Headline counts not independently re-derived.** The 22,069/21,898/118/2 figures and the "667 pages / ~10,000 events" figure come from dashboard tiles and the author's own working draft, not from a `grep -c` / `uniq -c` pass over the raw `alerts.json` file.
- **Two pieces of evidence not supplied for this rebuild.** A Splunk dashboard pie-chart panel ("SOC Brute Force Detection", Failed Logon By Source IP) and a raw Splunk event-detail page (showing the failure reason and sub-status codes for one 4625 event) are described in the author's working draft but were not among the images provided when this version of the report was built, so they are reported as described, not shown as figures.
- **Splunk query results not captured.** The SPL query in Chapter 7 is reported as written and run, but its actual result rows (which source IP(s), how many failures) were not supplied as evidence here.
- **No automated response was tested.** The attack was allowed to continue to a successful logon; no lockout, block, or disable action was configured or evidenced.
- **Single attacker, single account, one network path.** The Wazuh-vs-Splunk grouping-key difference identified in Finding F5 (account vs source IP) was not tested against a distributed-IP or password-spray attack pattern that would actually distinguish the two detections.
- **Sigma correlation portability untested against an alternative backend.** It is known to fail against this lab's OpenSearch/Lucene backend; whether a different backend or pipeline would succeed was not tested here.
- **Two IP addresses recorded for the same host** (192.168.56.101 vs .102 for VM1) are carried through unresolved, as noted in Chapter 10.
- **Photographic evidence.** All figures are photographs of VirtualBox guest screens rather than native screen captures, with the associated glare, angle and resolution limits that implies.

Future improvements: implement and test the active-response mitigation (M5); re-derive the headline counts from the raw alert file; capture the missing Splunk dashboard and raw-event evidence; test a distributed/password-spray attack shape against both detections; and find a Sigma backend that supports correlation rules so the full two-stage detection can be shown converting end to end.

CHAPTER 17

Conclusion

The lab objective was met. A working brute-force detection pipeline was built, deployed and validated against a live, if simulated, attack: CrackMapExec's dictionary attack against the domain account HR-STAFF1 succeeded (password `Hope199872`), and Wazuh detected the attack end to end — 118 authentication-failure alerts and 2 authentication-success alerts were recorded for the target agent, with the custom correlation rule (100101: 5 failures / 120 seconds, MITRE T1110) escalating ahead of the successful compromise and a built-in rule (92652) confirming the matching successful logon. An independently built Splunk SPL query reproduces the same threshold logic, grouped by source IP rather than by account. The Sigma rule set made the single-event detection portable across backends and, in the process, surfaced a genuine and useful limitation: the installed OpenSearch/Lucene backend does not support Sigma correlation rules, so only the first stage of a two-stage detection like this one currently converts.

The report does not claim more than this rebuild's evidence shows. The headline alert counts and the Splunk query's result rows are reported as the author's working draft states them, not independently re-derived here; two pieces of evidence described in that draft were not supplied as images and are not presented as figures. The strongest verified finding is that the detection-to-investigation chain worked across two independent SIEMs for the core attack pattern tested; the clearest next step is closing the gap between detection and automated response, and re-testing Sigma's correlation portability against a backend that supports it.

Appendix A: Evidence Register

Table 18. Evidence register: every figure and its source (originals kept unchanged)

Figure	Title	Source file prefix	Processing
Figure 1	Laboratory architecture and data flow	Drawn for report	Original diagram
Figure 2	Kali terminal: wordlists, housekeeping and the CrackMapExec launch	Kali desktop photo, same session continues in Figure 2b	Cropped, contrast-adjusted, sharpened
Figure 3	Early attack output: non-existent accounts rejected at the connection level	Reused from the lab's original drop-path screenshot set; timestamp not independently legible	Cropped, contrast-adjusted, sharpened
Figure 4	Continued cycling through non-existent accounts	Kali desktop photo	Cropped, contrast-adjusted, sharpened
Figure 5	Attack run reaching the end of the non-existent-account list	Kali desktop photo	Cropped, contrast-adjusted, sharpened
Figure 6	Wazuh local_rules.xml open in nano: the built-in rules above the custom ones	Ubuntu terminal, nano 8.7.1	Cropped, contrast-adjusted, sharpened
Figure 7	Rules 100100 and 100101 as deployed	Ubuntu terminal, nano 8.7.1	Cropped, contrast-adjusted, sharpened
Figure 8	Rule 100101 scrolled fully into view, with its closing tags	Ubuntu terminal, nano 8.7.1	Cropped, contrast-adjusted, sharpened
Figure 9	Sigma correlation-rule conversion failing, and the single-event rule succeeding	Kali terminal, two sessions (13:28 and a later one at 8:58) shown together	Cropped, contrast-adjusted, sharpened
Figure 10	Event log: the escalation pattern, page 2 of 667	Wazuh dashboard, Oct 5, 13:25	Cropped, contrast-adjusted, sharpened
Figure 11	Event log: earliest visible page, with the 4625/4624 context	Wazuh dashboard, Oct 5, 13:24	Cropped, contrast-adjusted, sharpened
Figure 12	Dashboard overview: alert-group evolution and alert-level timeline	Wazuh dashboard, Oct 5, 13:26	Cropped, contrast-adjusted, sharpened
Figure 13	Dashboard summary tiles: the headline counts	Wazuh dashboard, Oct 5, 13:26	Cropped, contrast-adjusted, sharpened

Figure	Title	Source file prefix	Processing
Figure 14	Top 5 alerts, rule groups and PCI DSS requirements	Wazuh dashboard, Oct 5, 13:27	Cropped, contrast-adjusted, sharpened

Appendix B: Full Command Reference (Author's Step-by-Step Replication Log)

Verified. This is the author's own numbered run log (Steps 1–34), reproduced here as a single reference block because it is the most literal reproduction evidence available: it was written at the time, numbered as executed, and is reused verbatim rather than re-derived. Narrative context for each step is in Chapter 6.

```
Step 1 - Build the wordlists (Kali).
nano usernames.txt      # 1,000+ entries
nano passwords.txt      # 1,000+ entries

Step 2 - Install CrackMapExec and confirm connectivity (Kali).
sudo apt update
sudo apt install crackmapexec -y
crackmapexec --version
ping 192.168.56.116

Step 3 - Check the Ubuntu IP (VM1).
ip addr                 # -> 192.168.56.102, host-only interface

Step 4 - Check the OS version (VM1).
lsb_release -a

Step 5 - Check Wazuh services (VM1).
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer

Step 6 - Check Splunk (VM1).
sudo systemctl status splunk
ls /opt/splunk/bin/splunk    # if the service name isn't found

Step 7 - Verify connectivity to the other VMs (VM1).
ping -c 4 192.168.56.115     # Windows Server
ping -c 4 192.168.56.116     # Windows 10 client

Step 8 - Start Wazuh and Splunk (VM1).
sudo systemctl start wazuh-manager
sudo systemctl status wazuh-manager
sudo /opt/splunk/bin/splunk start
sudo /opt/splunk/bin/splunk status

Step 9 - Open Splunk in the browser (VM1).
http://192.168.56.102:8000

Step 10 - Open the Wazuh dashboard.
https://192.168.56.102:443

Step 11 - Open PowerShell as Administrator (Windows 10/VM2).
Start -> search "PowerShell" -> right-click -> Run as Administrator.

Step 12 - Check the Windows IP (VM2).
ipconfig

Step 13 - Test connectivity to Wazuh (VM2).
ping 192.168.56.102

Step 14 - Check the agent communication port (VM2).
Test-NetConnection 192.168.56.102 -port 1514

Step 15 - Check the enrollment port (VM2).
Test-NetConnection 192.168.56.102 -port 1515
# Both return TcpTestSucceeded: True when reachable.

Step 16 - Check/start the Wazuh agent service (VM2).
Get-Service "WazuhSvc"
Start-Service WazuhSvc

Step 17 - Create the Sigma project structure (Kali).
mkdir -p Sigma_project/rules

Step 18 - Install Sigma prerequisites (Kali).
sudo apt install git python3 python3-venv pipx -y
```

```
Step 19 - Clone the official Sigma rules (Kali).
git clone https://github.com/SigmaHQ/sigma.git

Step 20 - Confirm the files are present (Kali).
ls

Step 21 - Install the Sigma CLI (Kali).
pipx ensurepath
pipx install sigma-cli
sigma --version

Step 22 - Install the Sigma backend plugins (Kali).
sigma plugin install splunk
sigma plugin install wazuh
sigma plugin install opensearch
sigma plugin install sysmon

Step 23 - Verify the Sigma install (Kali).
sigma plugin list
sigma list targets
sigma list pipelines
sigma list formats opensearch

Step 24 - (Optional) clone the official rules to study (Kali).
git clone https://github.com/SigmaHQ/sigma.git ~/sigma-official
# Note: cloning into the same directory as the working rules overwrites
# any pre-installed Sigma rules - clone to a separate path as shown.

Step 25 - Run the brute-force attack against HR-STAFF1 (Kali).
crackmapexec smb 192.168.56.116 -u usernames.txt -p passwords.txt -d HopeLabs.Local

Step 26 - Write the Sigma rule (Kali).
cd Sigma_project/rules
nano Brute_Force_SMB.yml
# (rule content: see Chapter 7)

Step 27 - List the OpenSearch Lucene pipelines (Kali).
sigma list pipelines opensearch_lucene
# Returns empty by default - see Step 28.

Step 28 - Create the custom pipeline and convert the rule (Kali).
nano wazuh_pipeline.yml
# name: wazuh-windows-fields
# priority: 20
# transformations:
#   - id: field_map
#     type: field_name_mapping
sigma convert -t opensearch_lucene -p wazuh_pipeline.yml Brute_Force_SMB.yml

Step 29 - Build the Splunk query (Splunk Search & Reporting).
index=* EventCode=4625
| rex "(?i)Source Network Address.*?(?<src_ip>\S+)"
| stats count as failed_attempts by src_ip
| where failed_attempts >= 5
| sort -failed_attempts

# Alternative (counts only, no threshold filter):
index=* EventCode=4625
| stats count

Step 30 - Build the Wazuh query (Wazuh Discover).
data.win.System.EventID: "4625"

Step 31 - Write the Wazuh detection rules (VM1).
sudo nano /var/ossec/etc/rules/local_rules.xml
# (rule content: see Chapter 7)
```

```
Step 32 - Validate the new rule (VM1).
sudo /var/ossec/bin/wazuh-analysisd -t
# A successful run prints no errors.

Step 33 - Confirm the alert in the Wazuh dashboard.
# Wazuh dashboard -> Threat Hunting -> Events - filter or scroll
# to "Multiple Failed Logons - Possible Brute Force".

Step 34 - Confirm the alert in Splunk.
# Splunk -> Search & Reporting -> Search:
index=* EventCode=4625
# or, to reproduce the thresholded view from Step 29:
index=* EventCode=4625
| rex "(?i)Source Network Address.*?(?<src_ip>\S+)"
| stats count as failed_attempts by src_ip
| where failed_attempts >= 5
| sort -failed_attempts
```

Appendix C: Glossary

Term	Meaning
Agent / forwarder	Software on an endpoint that collects logs and sends them to a central SIEM (the Wazuh agent and the Splunk Universal Forwarder are both examples)
Alert	A record written when a rule or query condition matches
Correlation rule	A rule that fires when another rule or event type matches several times within a time window
Dictionary attack	A brute-force attack that tries passwords from a prepared wordlist rather than every possible combination
Logsource	In Sigma, the description of where an event comes from (product, service, category) that a rule's detection logic is written against
NTLM	A Windows challenge-response authentication protocol, visible here in the "Successful Remote Logon Detected … NTLM authenticat…" alert
Pipeline (Sigma)	A set of transformations (such as field-name mapping) applied when converting a Sigma rule to a specific backend's query language

References. MITRE ATT&CK technique T1110 (Brute Force), attack.mitre.org. Wazuh documentation: rule syntax (`if_sid`, `if_matched_sid`, `frequency`, `timeframe`) and `wazuh-analysisd -t`. Splunk documentation: SPL `rex`, `stats`, `where`. SigmaHQ: github.com/SigmaHQ/sigma, and the Sigma CLI backend-plugin documentation for correlation-rule support. SecLists (username/password wordlists): github.com/danielmiessler/SecLists. CrackMapExec: used here for the controlled SMB dictionary-attack simulation against the HR-STAFF1 account.